Final Technical Report: YCP Hacks Website

Evan Natale, Logan Kinzler, Ethan Nelson

Abstract

Our clients, Evan Natale, Logan Kinzler, and Oscar Winters, play a crucial role in organizing and running York College of Pennsylvania's annual hackathon event. YCP Hacks is a three-day event held in the Willman Business Center that brings students together to collaborate, design, and build innovative projects. The organizers currently rely on a combination of third-party tools and manual processes to handle registration, team formation, event communication, judging, and administrative coordination. These tools lack cohesion, flexibility, and reliability, resulting in inefficiencies for both participants and event staff.

To address these challenges, this project continues the work of last semester's YCP Hacks capstone team to develop an integrated, in-house hackathon management platform for YCP Hacks. The system provides a centralized web application that supports participant registration, team creation, event scheduling, and organizer-level administrative tasks, among other capabilities. Our work for the Spring 2026 semester builds on the foundation established by last semester's team by extending the platform into a more complete content management system (CMS), implementing security measures, and configuring the platform for deployment to allow public access to the application. The result is a more reliable, flexible, and scalable solution that improves the management of current and future YCP Hacks events.

Introduction

As a team, we were given the opportunity to continue the development of the YCP Hacks web application, an in-house platform designed to support attendees and organizers during York College of Pennsylvania's annual three-day hackathon. Last semester's capstone team continued development on the application, replacing most hard-coded frontend values and connecting those components to the backend, implementing a hardware management interface for both participants and staff, as well as team registration functionality on the admin page with a page on the participants view showing them info about the team participants that have been registered to. While the previous team helped to further improve the existing project, many changes and additions are needed before this application to be the primary tool used to run the hackathon. In its current state, the only security measures implemented are page redirects, and the application is not currently configured to work in a deployment environment.

The primary focus was to expand and strengthen the existing application by reshaping it into a more complete and functional CMS.

- ➢ Implement protected routes to prevent unwanted access even with direct API request
- ➢ Configure the entire application to function in a deployment environment
- ➢ Create categories and prizes pages for staff to manage the displayed hackathon prizes that can be won and what categories each prize is in
- ➢ Build an email verification system that requires all users to verify the email address used to create an account
- ➢ Implement a password reset system that allows users to update their existing password
- ➢ Overhaul the participants site user interface to have consistent format across all pages and a light and dark mode toggle
- ➢ Mobile support for the participants site

These accomplishments resulted in the platform directly supporting core event-management tasks and providing a more cohesive, flexible, and scalable solution for YCP Hacks.

This report details the design, implementation, and evaluation of the continued development of the YCP Hacks platform. It outlines the relevant technical background, describes

the system's architecture and major components, discusses key implementation challenges, and identifies opportunities for future improvements by subsequent capstone teams.

Background

The YCP Hacks web application is designed to provide a centralized, in-house platform for managing York College of Pennsylvania's annual hackathon. The long-term objective of this multi-year project is to create a system capable of supporting both participants and organizers throughout all phases of the event. Ultimately, the platform should streamline the planning and execution of the hackathon by consolidating information and reducing organizers' dependence on external tools. Furthermore, a key long-term requirement is to allow for archival of past events for historical reference and institutional knowledge preservation, enabling organizers to quickly retrieve data from previous years to inform planning for future hackathons.

When complete, the YCP Hacks platform will include functionality such as:

- Public event information for participants
- Attendee account creation and registration
- Team formation and team management
- Hardware listings and checkout workflows
- Hardware inventory management for staff
- Event scheduling and activity tracking
- Administrative audit logs and oversight tools
- Project submission workflows
- Judging coordination
- Automated event announcements and messaging
- Sponsor and prize information management
- Mobile device support
- Automated site deployment

The 2024 capstone team developed the first operational version of the current YCP Hacks platform using Express.js, Vue.js, Docker, and MySQL. Their contribution established the project's initial architecture and provided the core functionality for participant registration, account creation, basic event navigation, and staff login. Several administrative pages were created during that year as well, although many relied on hard-coded placeholder values rather than real data from the backend.

This version of the project was then continued by the 2025 capstone team using the same platform. This team's contributions cleaned up the site by replacing most of the hard-coded frontend values and connecting those components to the backend. More administrative and participants pages were added, as well as automated back end testing, and a reconstructed and improved data model for better organization and efficiency.

This semester's capstone team has expanded upon that foundation by improving security measures with protected routes, building out the remaining essential pages, and to configure and set up the application in an active deployment environment. These enhancements transformed the system from its previous state into a more complete and maintainable CMS that can better support the operational needs of YCP Hacks organizers.

Technical Terms

Content Management System (CMS) - a system that allows administrators to create, edit, and manage content on a website without modifying source code directly.

CRUD - stands for create, read, update, and delete. These are the four fundamental operations performed on data in a persistent database.

Environment variable - a configuration value stored outside the application code that influences program behavior (e.g., API keys, database credentials, mode settings).

Framework - a collection of tools, libraries, and conventions that simplify and accelerate software development by providing reusable components and structured patterns.

Integrated Development Environment (IDE) - a software application that provides tools for writing, editing, testing, and debugging code in one comprehensive interface.

Role-Based Access Control (RBAC) - a security model that restricts system access to authorized users based on their roles within an organization.

Web Application - software accessed through a web browser that interacts with backend services or databases to provide dynamic functionality.

Continuous Integration/Continuous Deployment (CI/CD) Pipeline - an automated process of building, testing, and deploying software.

Services Used

DigitalOcean

DigitalOcean [7] is an application hosting and cloud platform used to deploy the project's backend, frontend, and database systems. It provides scalable infrastructure for running the platform in a production environment. DigitalOcean was chosen due to its low overhead cost, straightforward deployment pipeline, and support for Docker-based applications.

GitHub

GitHub [1] serves as the platform for version control, code storage, and collaborative development. It allows multiple developers to work on different parts of the system simultaneously while maintaining a shared and organized repository. GitHub also integrates with DigitalOcean for future CI/CD deployment pipelines.

Docker

Docker [11] is used to containerize both the development and production environments, ensuring consistency across systems and simplifying deployment. Containers allow the team to run MySQL, backend services, frontend services, and supporting tools in isolated, reproducible environments.

GitHub Container Registry (GHCR)

GitHub Container Registry [20] is used to store container images. This provides a place the droplet can pull the back end and front end docker container images.

MySQL

MySQL [30] is an open source relational database management system that is used to store and manage data. Data is stored in tables of rows and columns organized into schemas.

my-sql-cron-backup

my-sql-cron-backup [22] is a docker image that runs mysqldump to locally backup the data in the database periodically using the cron task manager. This provides a quick way to repopulate the database if it has to be rebuilt during development or the data is lost.

Wiki.js

Wiki.js [14] is used for documentation management throughout the project. It provides a Markdown-based editor, a clean interface, and collaborative features that make it easy for team members to share knowledge and maintain organized technical documentation.

GitBot

GitBot [31] is used to send notifications to Discord channels whenever a push occurs on a branch in the Github repositories and submodules. It provides a way to monitor the changes to the project during development and deployment.

Nodemon

Nodemon [32] is the development monitoring tool used to detect changes within the project and automatically rebuild the project with the current development environment. This is used to increase development efficiency and speed while making small, incremental changes to the project.

Vite

Vite [9] is the frontend development tool that optimizes the build-time of web applications during runtime. This provides efficiencies in building the project during development.

Tools Used

Node.js (v16.0.0)

Node.js [8] is an asynchronous, event-driven JavaScript runtime environment used to build the backend API and server-side logic of the YCP Hacks platform.

npm (v10.2.1)

The Node Package Manager [15] is used to install and manage project dependencies for both backend and frontend components of the system.

Bootstrap (v5.38)

Bootstrap [4] provides a collection of prebuilt components, responsive grid utilities, and styling tools. While custom styling is used extensively, Bootstrap supports rapid UI development and ensures consistent visual design across pages.

dotenv (v16.4.5)

dotenv [10] loads environment variables from a .env file into the running application. This separates configuration from code, making deployments more secure and easier to manage.

Axios (v1.13.5)

Axios[5] is a JavaScript package used to make asynchronous HTTP requests. This provides the communication from the frontend to the API.

Express.js (v4.19.2)

Express.js [17] is the primary backend framework used to define API endpoints, route requests, and interact with the database layer. It provides a flexible and lightweight foundation for the server.

validator.js

validator.js [25] is a Node.js package that contains a library of string validators and sanitizers. These are utilized to clean data in incoming requests to prevent invalid or harmful data from reaching the server.

Express-validator(v7.3.1)

Express-validator [24] is a set of express.js middleware that wraps validator.js to provide a clean solution to chain validation and sanitization for incoming requests.

Vue.js (v3.4.29)

Vue.js [2] is the frontend JavaScript framework used to develop dynamic, reactive components for both attendee-facing and admin-facing pages. It allows the platform to update content in real time based on data retrieved from the backend.

Vue Router (v4.4.5)

Vue Router [18] is the official routing library for Vue.js. It enables client-side navigation within the application by mapping URLs to Vue components. This allows the system to provide a seamless single-page application (SPA) experience while organizing attendee-facing and admin-facing pages in a structured and maintainable way.

Vuex (v4.1.0)

Vuex [19] is a state management library used to store and manage global application state across components. It provides predictable state mutations, centralized data access, and improved maintainability. For this project, Vuex stores information such as user authentication status, event details, and other shared state needed across multiple pages.

Vitest(v3.2.4)

Vitest [21] is a testing framework built on Vite, designed to integrate with JavaScript and TypeScript projects. This is used for implementing unit testing on both front ends of the application.

Jest(v29.7.0)

Jest [34] is a JavaScript testing framework that is used for back end unit testing.

qrcode(v1.5.4)

qrcode [23] is an npm package that is used to generate QR codes. This allows for information to be converted into a scannable QR code.

Sequelize (v6.37.3)

Sequelize [33] is the Object-Relational Mapping (ORM) tool used to interact with the MySQL database. It simplifies querying, schema definition, migrations, and model associations within the Express.js backend.

NodeMailer (v8.0.2)

NodeMailer[26] is the email-sending tool used to send emails directly from the backend to the email provided. NodeMailer creates a transporter, defines the email, then sends it.

Stream (v0.0.3)

Stream [27] is the object creator for CSV imports. It takes the csv information that is imported and creates it into an array of objects which will be added to the database. Currently, it creates a Readable from the csv data which means it reads the data from the csv, then creates the array.

CSV Parser(v3.2.0)

CSV Parser [28] is the tool used to map csv column headers to object keys which then allows Stream to create the proper objects to place in the database.

Puppeteer(v24.37.5)

Puppeteer [29] is used to control a web browser, Chromium in this project. Allows for pdf generation by the creation of a webpage then downloading the webpage to a .pdf.

Design

High-Level System Design

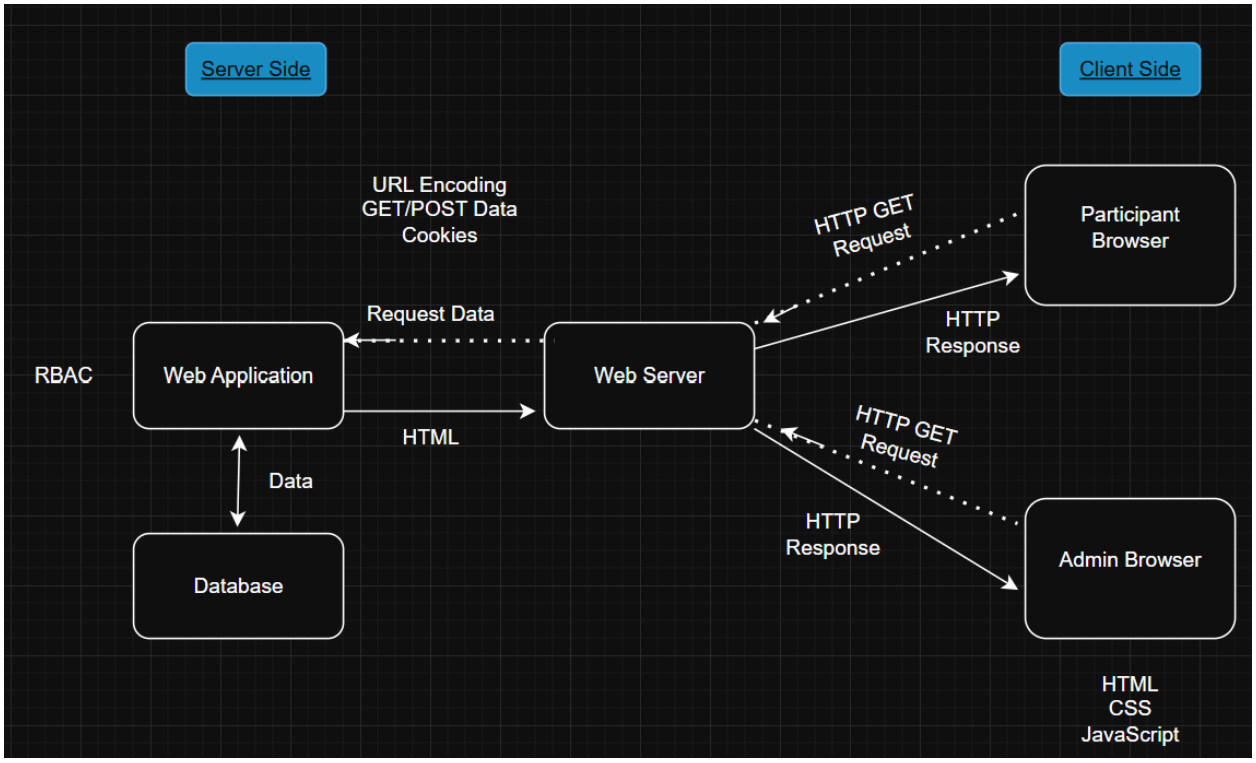


Figure 1. Architecture Diagram

Figure 1 illustrates the overall system architecture for the YCP Hacks web application, adopting a standard three-tier design. This architecture logically separates the system into the Client Side and the Server Side, ensuring a clear division of responsibility for scalability and maintainability.

System Components

The Client side encompasses the user interface for the application. This initiates the process by issuing HTTP GET/POST Requests and processes HTML, CSS, and JavaScript returned in the HTTP Response to render the view.

The Server Side handles all application logic, including the Web Server, Web Application, and Database.

The Web Server acts as the entry point and routing mechanism as it receives all client requests, handles the delivery of files, and forwards dynamic requests to the Web Application. It also formats the final HTTP Response back to the browser.

The Web Application is the core application logic tier. It executes the business logic, performs form validation, enforces RBAC, and communicates with the data persistence layer.

The Database is the persistence layer that stores and manages all application data. The Web Application interacts with it to query or update data, often using an ORM (e.g., Sequelize) to map relational structures.

System Level Interaction Flow

At a high level, the system follows a standard request-response cycle:

1) **Client Request:** Users interact with the application through a browser, which issues an HTTP GET or POST request to the Web Server for the desired page or API resource.
2) **Web Server Routing:** The Web Server examines the incoming requests. Requests for dynamic content are forwarded as Request Data to the Web Application.
3) **Backend Application Processing:** Dynamic API requests are handled by the backend. This layer processes the business logic, performs necessary validations, and executes access control checks. It then interacts with the Database to retrieve or modify persisted data.
4) **Response Generation:** After processing, the Web Application generates the updated data to rendered content (like HTML) and sends it back to the Web Server.
5) **Response to Client:** The Web Server then sends a comprehensive HTTP Response to the participant or admin browser, which is then rendered as the updated user interface.

To illustrate this flow, consider an Admin user checking the details of a registered team: The transaction begins with the Client Request, where the Admin's browser sends an HTTP GET Request (e.g., /teams/:id). The Web Server Routing forwards this request to the backend. During Backend Application Processing, the Controller first verifies the Admin's authorization, then delegates the task to the Service layer, which in turn calls the Repository to retrieve the team data from the Database. Upon receiving the data, the backend performs Response Generation, creating a JSON object containing the team details. Finally, the Response to Client step sees the Web Server send the HTTP Response back to the browser, which renders the team's information on the Admin interface.

Role-Based Access Control (RBAC) Implementation

The system architecture features Role-Based Access Control (RBAC) enforced by the Web Application to secure administrative functions and control user capabilities. RBAC ensures that access rights are granted or restricted based on the user's defined role, thereby adhering to the principle of least privilege.

For the YCP Hacks application, three distinct user roles have been defined to manage varying levels of operational access and data manipulation: Participant, Staff, and Oscar Level.

1) **Oscar Level (Highest Administrative Access)**

Users at Oscar Level (typically the organizational president or Oscar himself) possess full administrative oversight of both the participant and administrative sides of the application. These users have the ability to add or delete records pertaining to any page. However, this level is not allowed to add or delete accounts, as all users must be created through completion of the standard registration process. Users at the Oscar Level also have full permission to update or edit any record across the entire application, enabling real-time adjustments during the event or for future planning.

2) **Staff Level (Elevated Operational Access)**

Staff Level Users are granted elevated privileges necessary for event operations but have restrictions compared to the Oscar Level. Users at this level can add data to various application sections and edit or update existing records. Staff users are not allowed to delete any records preventing potential data loss.

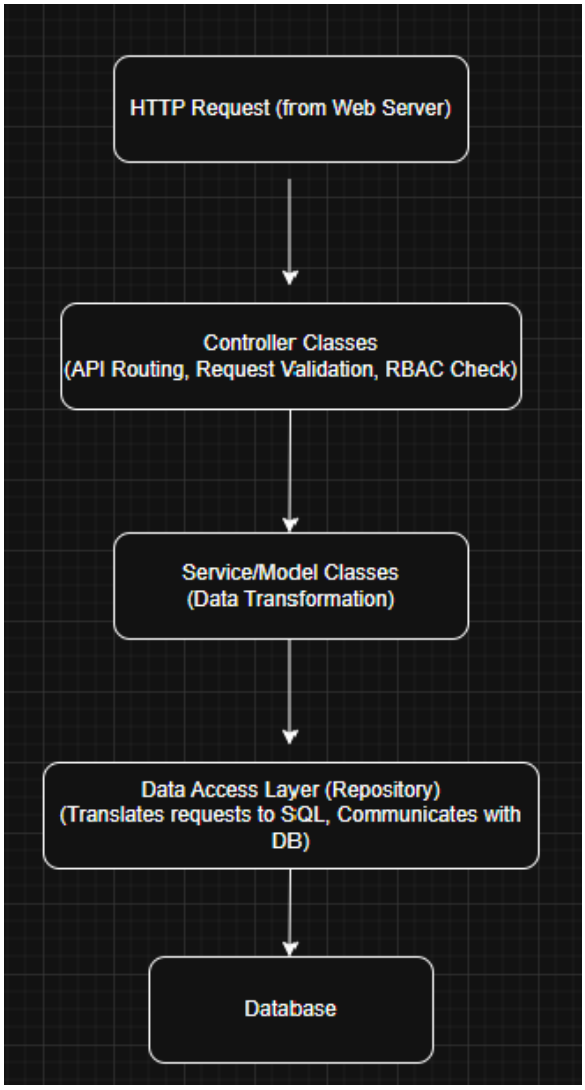
3) **Participant Level (Base User Access)**

Participants are standard users with access limited to the public-facing areas of the application. They have no access to the dedicated Admin webpage. With that, participants are not able to log in to the Admin page or change system wide settings. Users are able to view relevant event information linked from the participant webpage, such as Activities, Categories and the associated Prizes, and Hardware resources, which are typically also posted in the YCP Hacks

Discord server. The participant can only update their team information once they have successfully joined or formed a team. While they can't update their Team Name, they are able to update information based on their project details.

This architecture ensures a clear separation between the frontend interface, backend logic, and data storage. The modular design allows individual components to evolve independently and supports long-term maintainability for future capstone teams.

Web Application Internal Flow Diagram



The Web Application component, identified in the high-level architecture (Figure 1), is structured using a multi-layered design to process dynamic requests and enforce the separation of concerns. This internal flow, illustrated in Figure 2, shows the precise progression of a request from the Web Server to the Database.

The process begins when an HTTP Request is passed from the Web Server into the backend. It first encounters the Controller Classes. This layer is responsible for API routing, basic request validation, and initiating the RBAC check to confirm user authorization. The request is then passed to the Service/Model Classes, which house the application's core business logic and perform necessary data transformation. Following execution of the business logic, the request proceeds to the Data Access Layer (Repository). This layer is responsible for translating the application-level requests into specific SQL queries and communicating directly with the Database persistence layer. This tiered approach within the backend ensures that business logic remains independent of data access methods, making the application easier to maintain, test, and adapt.

Figure 2. Web Application Internal Flow Diagram

For example, to illustrate this flow, consider a user attempting to view their own profile information via the API: An HTTP GET request to the endpoint /users/profile/:id first hits the Controller Classes. The Controller verifies user authorization (RBAC Check) and delegates the task to the Service Layer by calling a method like userService.getProfile(userId). The Service/Model Classes then execute any business rules before calling the Data Access Layer (Repository) method, such as userRepo.findById(userId). The Repository translates this into a specific SQL query, retrieves the data from the Database, and returns it through the Service back to the Controller. Finally, the Controller formats the result into the final HTTP response (e.g., a JSON object with a 200 OK status) for the client.

UML Diagram

Figure 3. UML Diagram

The comprehensive UML Class Diagram (Figure 3) serves as the detailed blueprint for the YCP Hacks application's Controller Layer, moving beyond a basic data model to illustrate

the architectural design of the public API interface. This inclusion of CRUD (Create, Read, Update, Delete) functions within the diagram signifies its purpose as a logical design document, detailing the functional responsibilities of the backend components dedicated to handling external requests.

Visual Language and Separation of Concerns

The diagram strictly focuses on defining the application's API contract and the corresponding Controller actions, made visually explicit through a color-coding scheme:

- Controller Methods (Green): These are the internal functions (e.g., createTeam(), updateTeam()) that the Controller uses to receive an external request. Their responsibility is to handle the incoming HTTP call, perform basic data parsing/validation, and delegate the core operational task to the underlying Business Logic Layer (not shown).
- API Routes (Purple): This defines the application's external interface or API contract. It lists the specific HTTP Verbs (POST, GET, PUT, DELETE) and the Endpoints (e.g., /teams/create, /teams/:id) that clients interact with.
- Modals/Data Models (Black): This color scheme is used to identify the fundamental Data Structures and Entities within the application, such as TeamRegistration properties. These sections define the required data shape for input and output, clearly separating the data definition from the functional logic.

Architectural Flow and Controller Role

The overall request flow dictates that a client request arrives at the Controller's purple Route, is handled by a Green Controller Function, and is then processed. The Controller classes (e.g., UserController, TeamController) thus act as the API endpoint layer, dedicated to:

1) Handling incoming HTTP requests
2) Parsing and validating basic data format based on the defined Modals
3) Mapping the request to the appropriate internal Controller function
4) Delegating the execution of business logic (to an external layer)
5) Formatting the processed result into the final HTTP response for the client

Central Entity Relationships and Core Links

The entire data model is structured around two central entities: the Event and the User. The Event entity acts as the primary hub for all hackathon-specific data, establishing crucial one-to-many relationships with nearly all major entities, including Activities and Prizes. This design is fundamental to ensuring that all data is properly scoped, which prevents the retrieval or mixing of data belonging to different hackathon instances. Similarly, the User entity serves as the root for all people interacting with the system, linking authentication and role data necessary for RBAC to every operation; specific roles like Staff and Participants are typically modeled as specialized subsets of this core entity.

Detailed Analysis of Core Data Entities

The YCP Hacks application's persistence layer is defined by a data model that enforces contextual integrity, primarily revolving around the central Event entity and its crucial associations. This model is protected by the RBAC system, which relies on the User entity to categorize individuals as Oscar, Staff, or Participant and restrict their access based on their assigned role.

Event Participants and Team Registration

The core participant management system is structured around the TeamRegistration and EventParticipant classes. The TeamRegistration entity handles project-specific data, including attributes like teamName, projectDescription, and githubLink. Its methods, such as createTeam() and updateTeamProjectDetails(teamId; int), reflect the full CRUD capabilities necessary for participants to manage their project teams. This table holds a one-to-many relationship with the EventParticipant junction table. The EventParticipant table is critical as it links the central Event table to the User table, managing the enrollment list. Its specific method, like getTeamForParticipant(userId: int) and assignParticipant(userId: int, eventId: int), highlight its role in orchestrating team formation and confirming that a specific User is officially enrolled in and assigned to a Team within the context of a particular Event.

Event Sponsors and Sponsor Tiers

The sponsor system is designed for tracking of both organizations and their specific contributions to an event. This is managed through the Sponsors, EventSponsors, and SponsorTier entities. The Sponsors table stores general organizational information (e.g., sponsorName, sponsorWebsite), while the Sponsor Tier table defines the hierarchy of contribution levels, using attributes like lower_threshold to determine which tier a sponsor qualifies for. The EventSponsors table acts as the essential junction table, linking the Sponsors to the Event table. This is necessary because it is the designation location for storing event-specific attributes, such as the actual sponsorship tier achieved and the specific contribution details for that year's hackathon, preventing these transient attributes from cluttering the permanent organizational record in the Sponsors table. This design is critical for maintaining historical accuracy: when a sponsor's tier or contribution changes, that update is recorded only in the EventSponsors record for the current or future event, ensuring previous events retain their original, historically accurate sponsorship details. The methods associated with this table, such as updateEventSponsor(eventId: int, sponsorId: int, tierId: int) and removeSponsor(eventId: int, sponsorId: int), clearly show the operational management required to link or unlink a sponsor from a specific event and update their tier status.

Hardware Management

The Hardware entity, alongside the HardwareImages table, manages the inventory and distribution of physical equipment available for the hackathon. A key functional aspect of this system is that the application is designed to retrieve and manage the hardware inventory to the current active Event entity. The Hardware table stores details like hardwareName and its functional status, and includes the whoHasIt attribute to track which user currently has possession of the equipment – a vital piece of operational data. The functional methods, such as assignHardware(hardwareId: int, userId: int) and getHardware(hardwareId: int), confirm that this class handles the real-time operational logistics of checking hardware in and out. The separate HardwareImages table uses a foreign key to link back to the Hardware entity, allowing multiple images to be associated with a single piece of equipment without increasing the size or complexity of the main hardware record. This separation ensures retrieval of inventory and visual documentation.

Evolution and Development Changes to the UML Class Diagram

The final UML Class Diagram (Figure 3) reflects a significant evolution from the initial design, a necessary process driven by the detailed implementation of the system throughout the semester. These changes primarily addressed two areas: synchronizing the data model with the database schema and expanding the functional complexity of the application logic layers.

A major focus of the project implementation was ensuring strict consistency between the conceptual data model presented in the UML and the physical database schema. This synchronization involved continually revising the UML to match the exact schema definitions created by the ORM, including updating attribute data types and verifying all foreign key constraints, especially those linking back to the central Event and user tables, to enforce data integrity at the persistence layer.

The most substantial change involved expanding the functional representation of the Controller and Service layers. The initial UML captured only the fundamental data entities, but as API endpoints were implemented, the diagram had to be expanded to reflect the full operational capability of the system. Service classes were expanded with numerous functions to encapsulate complex business logic, such as checking RBAC permissions and validating data. Concurrently, the Controller classes were expanded to include a full mapping of all necessary HTTP routes (e.g., GET /teams/all, POST /hardware/add, DELETE /sponsors/:id), confirming the final UML is an accurate design document detailing every API endpoint exposed by the backend and the specific Controller function responsible for handling it. This continuous refinement ensured the final UML Class Diagram is a current representation of the implemented system's architecture and capabilities.

Front End

The front end of the YCP Hacks web application is implemented as a single-page application (SPA) using Vue.js, supported by Vue Router for client-side navigation and Vuex for state management. The primary goal of the front-end design is to provide an intuitive, responsive, and consistent user experience for both hackathon participants and event organizers. Because of the project's scope and timeline, the team focused on developing the essential pages

and workflows required for the system to function as a complete content management platform, while prioritizing clarity and maintainability in the user interface.

Consistent with the previous team's decision, we did not create wireframes or mockups for the system. Again, the team relied on the existing design patterns established last year and expanded upon them to accommodate the significantly increased functionality introduced during this semester. By adopting and extending these pre-established layouts, we ensured consistent styling, navigation, and component structure across all new pages without needing formal wireframe artifacts. This approach allowed the team to allocate more time to implementing dynamic features, backend integration, and usability improvements across both participant and administrator experiences.

The system contains two major categories of front-end pages:

1. **Participant-facing pages**, which provide attendees with access to registration, team management, event information, and hardware listings.
2. **Administrator-facing pages**, which provide organizers with tools to manage events, users, hardware, teams, sponsors, activities, and system audit logs.

Each page described below includes its purpose, major design components, supporting data structures, and primary user inputs and outputs.

Landing Page

Purpose: Serves as the initial entry point for attendees and general visitors, providing high-level event information and easy access to registration and login (see Figure 4).

Design Components:

- Event highlights
- Quick links to register or log in
- Sponsor showcase
- Navigation bar for participant pages

Supporting Data Models:

- Event
- Activities
- Sponsor

Inputs/Outputs:

- Input: None
- Output: High-level event and activity information and sponsor display

Figure 4: Shows the top of the Landing page, where users can navigate directly to registration/login, or scroll down to view further information like planned activities and current sponsors.

Participant Registration Page

Purpose: Allows users to register for the hackathon and create an account within the system (see Figure 5).

Design Components:

- Registration form (name, email, password, optional team info)
- Terms and conditions acknowledgment
- Registration confirmation message

Supporting Data Models:

- User
- EventParticipant (created after registration)

Inputs/Outputs:

- Input: Registration form data
- Output: Confirmation of successful registration or validation errors

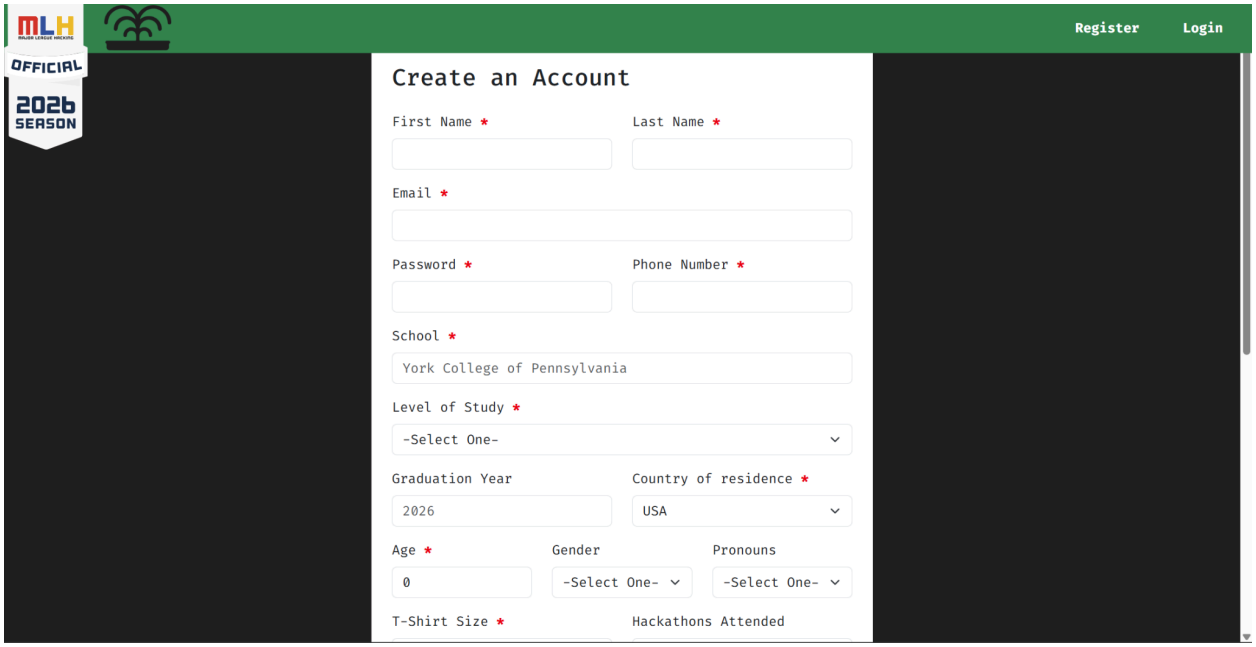


Figure 5: Shows the Registration Page that is MLH-compliant with a user-friendly form layout, a checkbox for agreeing to terms, and a submit button.

Participant Login Page

Purpose: Allows attendees to authenticate and access participant-specific pages (see Figure 6).

Design Components:

- Login form (email, password)
- “Forgot password” link
- Error and success messaging

Supporting Data Models:

- User

Inputs/Outputs:

- Input: Email and password
- Output: Login success or failure message

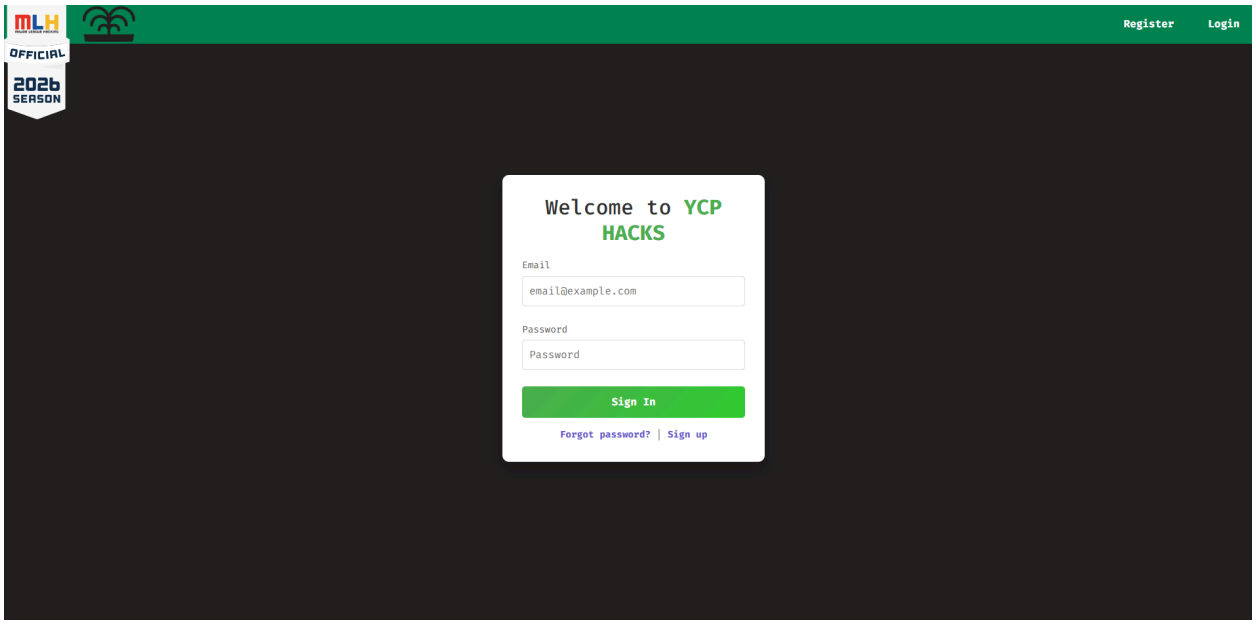


Figure 6: Shows the Admin Login Page with clearly marked email and password fields, a link for password recovery, and a submit button for login.

Check In Page

Purpose: Allows participants to view their QR code that can be scanned for checking into an event (see Figure 19).

Design Components:

- Display of user QR code

Supporting Data Models:

- User

Inputs/Outputs:

- Input: None
- Output: QR code

Figure 7. Shows the participants QR code that can be shown and scanned by staff to be checked into the event

Team Management Page

Purpose: Allows participants to view their team or update team information (see Figure 7).

Design Components:

- Display of team roster
- Ability to update team information

Supporting Data Models:

- Team
- EventParticipant

Inputs/Outputs:

- Input: Team update fields
- Output: Team details

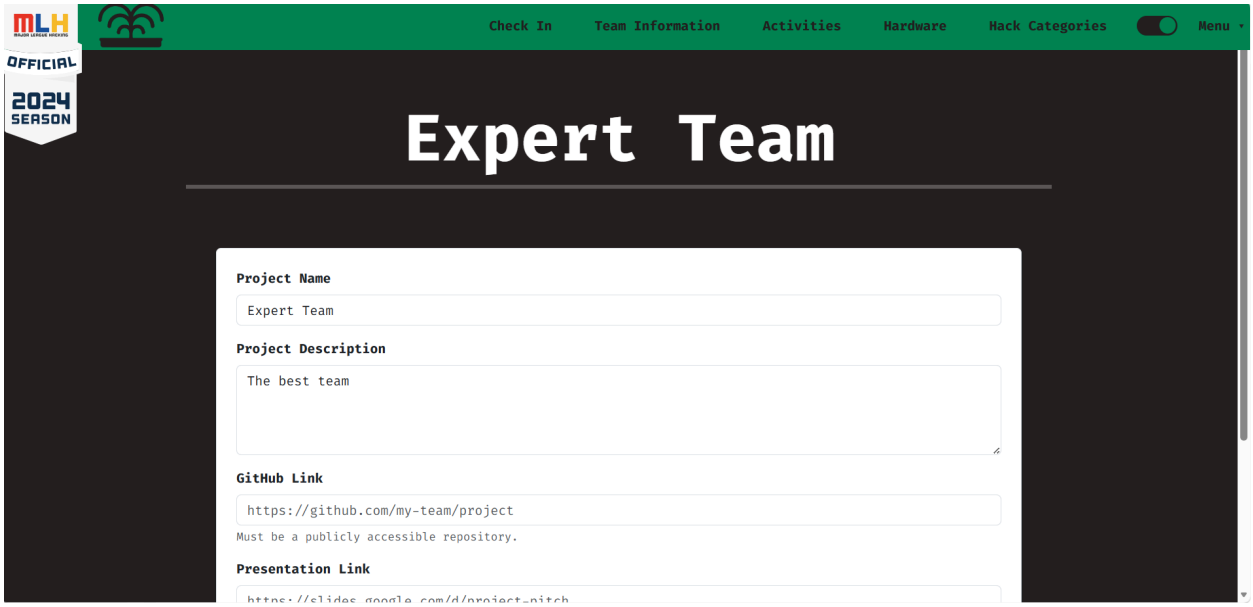


Figure 8: Shows the Team Project Page, where the Participants are able to update their project information for that event before the event ends.

Hardware Listings Page (Participant)

Purpose: Displays all hardware available for checkout by participants during the event (see Figure 9).

Design Components:

- Hardware table with search and filter capabilities
- Hardware details (name, description, quantity, availability)
- Images of hardware items

Supporting Data Models:

- Hardware

- HardwareImage

Inputs/Outputs:

- Input: None
- Output: Dynamic inventory display

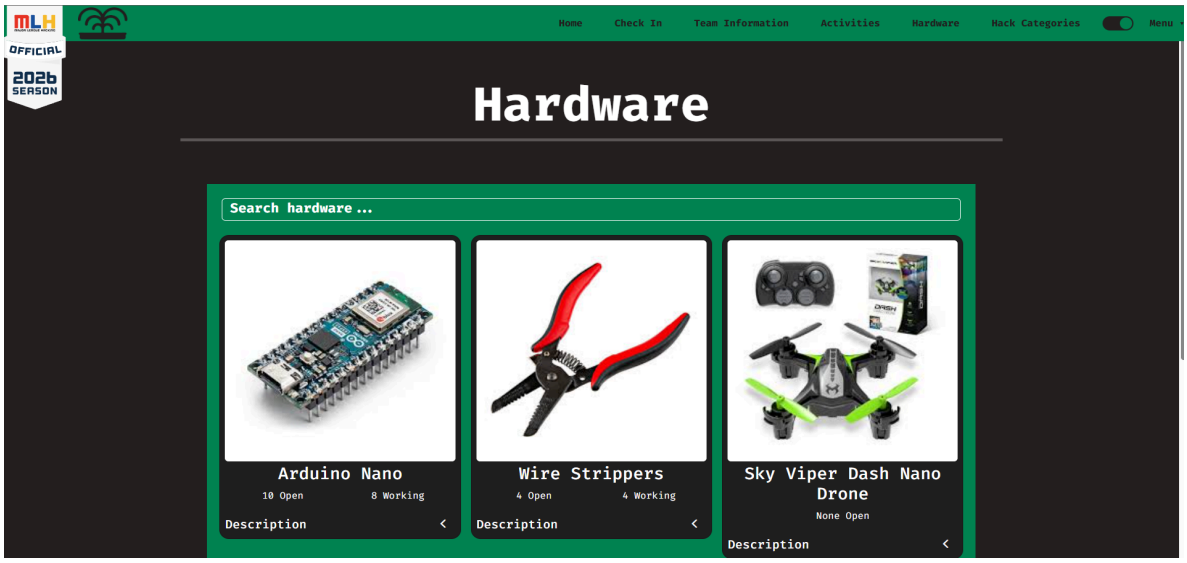


Figure 9: Shows the Hardware Display Page which lists all hardware added by staff, with relevant information displayed and a search bar for filtering.

Activities Page (Participant)

Purpose: Displays the schedule of hackathon activities for the event (see Figure 9).

Design Components:

- List of scheduled activities
- Activity descriptions and times
- Mobile-friendly timeline format

Supporting Data Models:

- Activity
- EventActivity

Inputs/Outputs:

- Input: None
- Output: Activity listings for the user's event

Figure 10: Shows the Activity section that displays for participants. All activities are listed out for the current active event and can be clicked on to display additional information.

Sponsors Section (Participant)

Purpose: Highlights event sponsors and sponsorship tiers (see Figure 11).

Design Components:

- Sponsor logos
- Sponsor logo size based off of tier
- Links to sponsor websites

Supporting Data Models:

- Sponsor
- EventSponsor
- SponsorTier

Inputs/Outputs:

- Input: None
- Output: Sponsor information and branding

Figure 11: Shows the Sponsors section of the Landing page, which displays all sponsors and their logo/name.

Hack Categories (Participant)

Purpose: Highlight the different types of categories in which participants can work in for their project.

Design Components:

- List of categories that participants can compete in
- List of prizes in each category

Supporting Data Models:

- Categories
- Prizes

Inputs/Outputs:

- Input: none
- Output: category information and prizes for each category

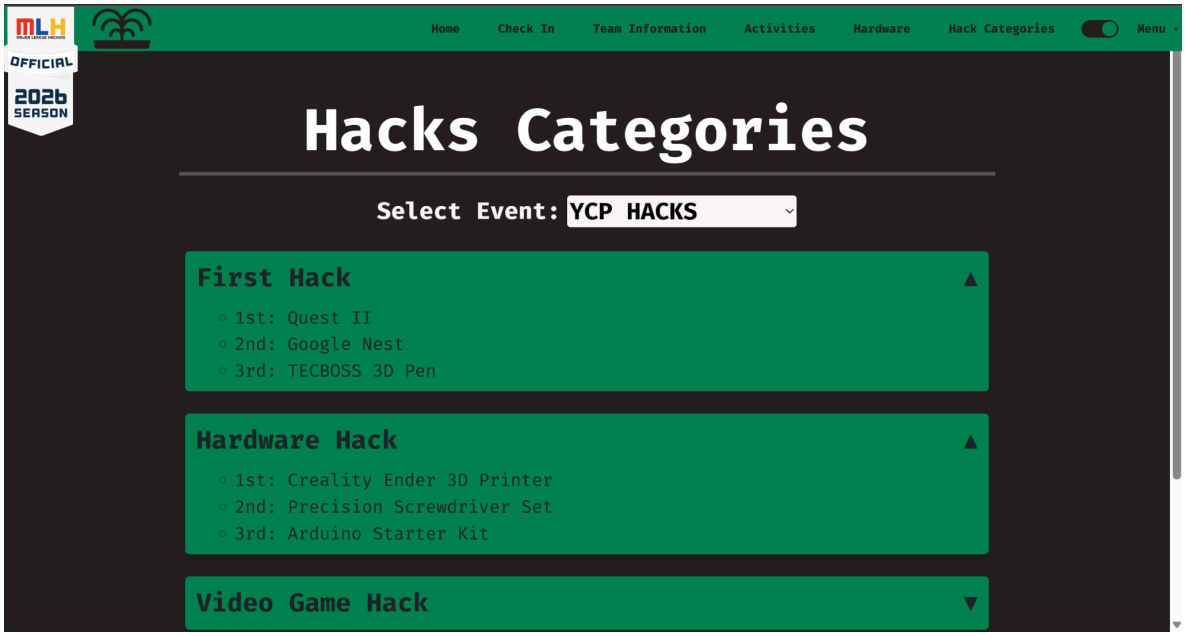


Figure 12: Shows the Hacks Category section of the webpage, located under the Hack Categories route of the navigation bar.

Email Verification (Participant)

Purpose: Location for the user to start the process of email verification.

Design Components:

- Button containing route which is send link to the signed in users email

Supporting Data Models:

- User

Inputs/Outputs:

- Input: User needs to click button which will activate the route
- Output: Button once clicked will switch display to show it is actively sending until route finishes

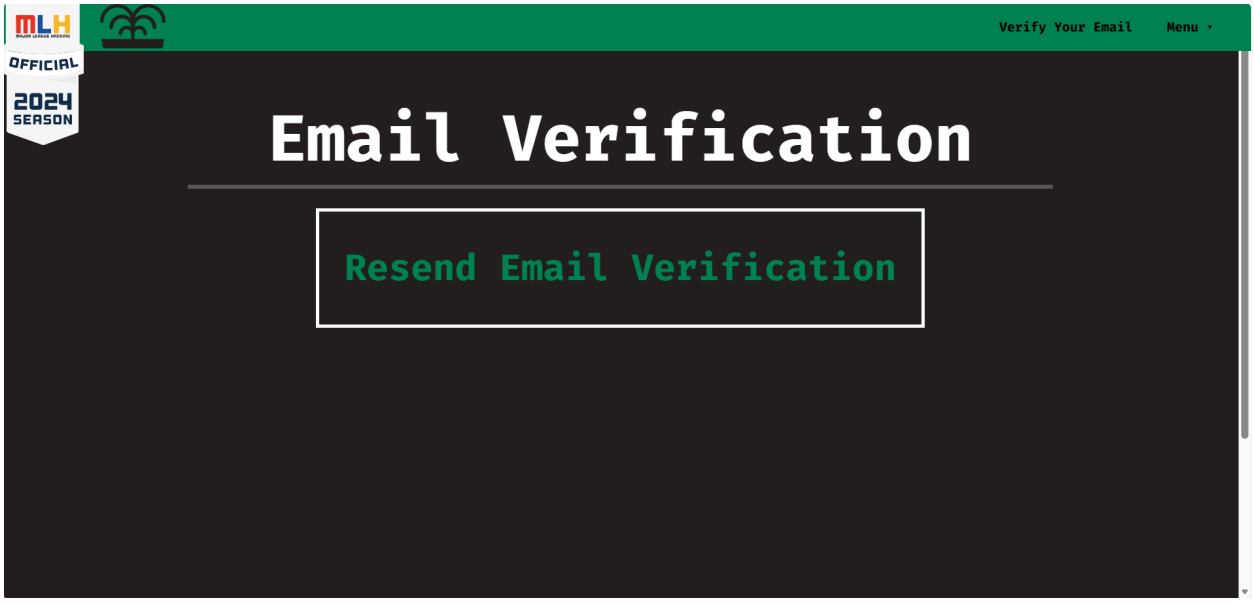


Figure 13: Email verification section of the website which is a button that will send the user their email verification email.

Profile Page

Purpose: Provides a page for participants to edit their biographical information, send password reset requests, and email verification requests.

Design Components:

- Biographical information fields
- Email and password field
- RBAC enforcement (redirects based on role permissions)

Supporting Data Models:

- User

Inputs/Outputs:

- Input: Updated biographical information and email
- Output: User's biographical information

Figure 14: Shows the current state of the Profile page with clearly labeled and filled biographical information

Admin Login Page

Purpose: Authenticates event staff and grants access to administrative tools (see Figure 11).

Design Components:

- Email and password fields
- Authentication messaging
- RBAC enforcement (redirects based on role permissions)

Supporting Data Models:

- User

Inputs/Outputs:

- Input: Login credentials
- Output: Dashboard access or error messages

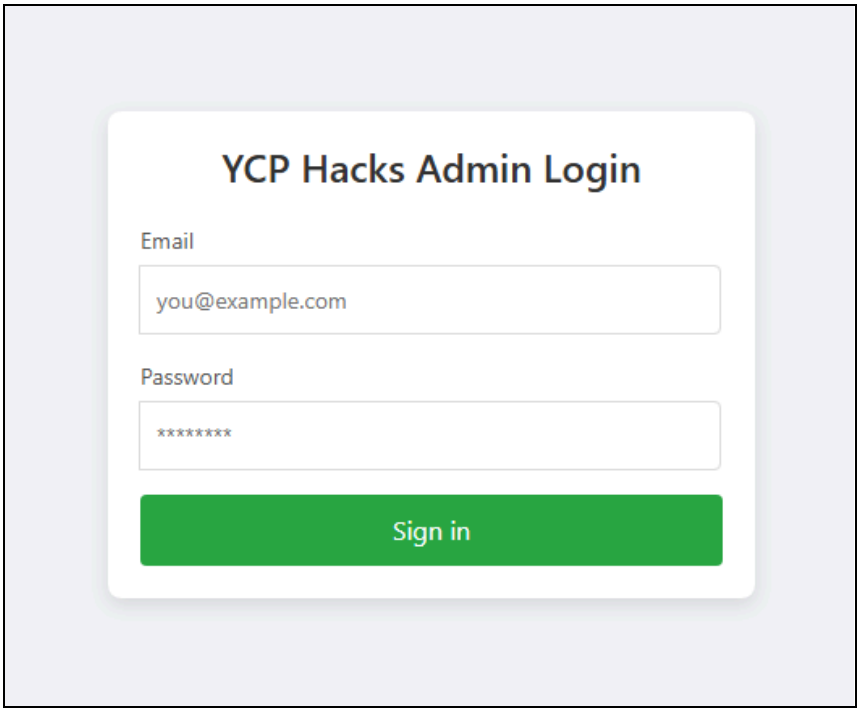


Figure 15: Shows the current state of the Admin Login page with clearly labeled username and password fields and a submit button for secure login.

Admin Dashboard

Purpose: Provides an overview of event metrics and quick access to administrative management tools (see Figure 16).

Design Components:

- Summary cards (e.g., total participants, hardware inventory counts)
- Upcoming activities
- Navigation sidebar for admin pages

Supporting Data Models:

- User
- Event
- Hardware
- Activity

Inputs/Outputs:

- Input: None
- Output: Overview of event data and system metrics

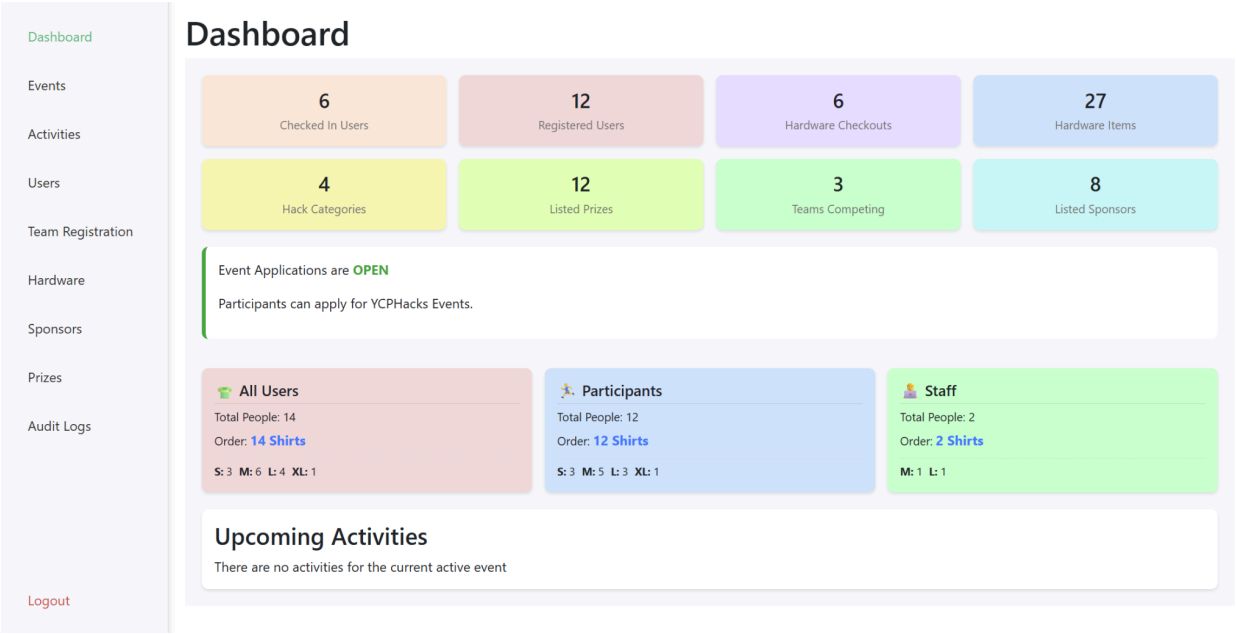


Figure 16: Shows the current state of the admin dashboard displaying metrics in overview cards (e.g., "Registered Users" and "Hardware Items"), a sidebar with navigation links to all the admin pages.

User Management Page

Purpose: Allows staff to view and edit user details (see Figure 13).

Design Components:

- User table with sorting and filtering
- User detail modal
- User check in via QR code scanner (see Figure 19)

Supporting Data Models:

- User
- StaffRole

Inputs/Outputs:

- Input: Search, edit roles, update permissions, update check in status
- Output: Updated user role information, updated check in status

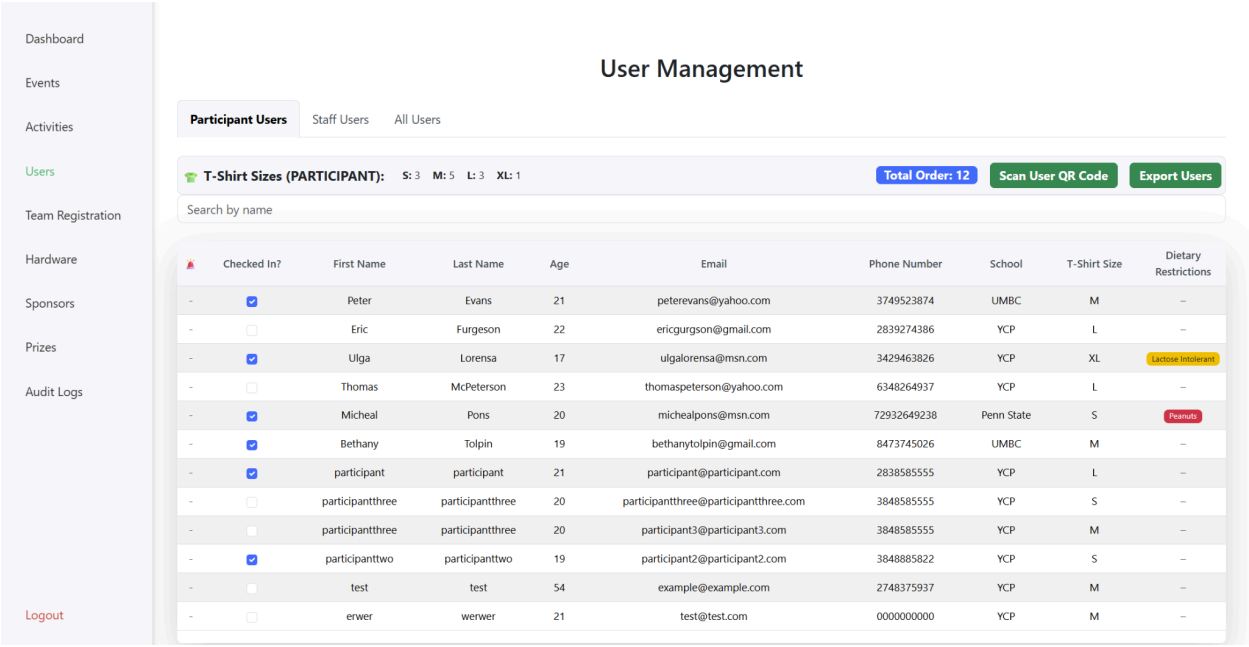


Figure 17: Shows the User Management Page which lists all users with various options to create, update, filter, export, or check in users.

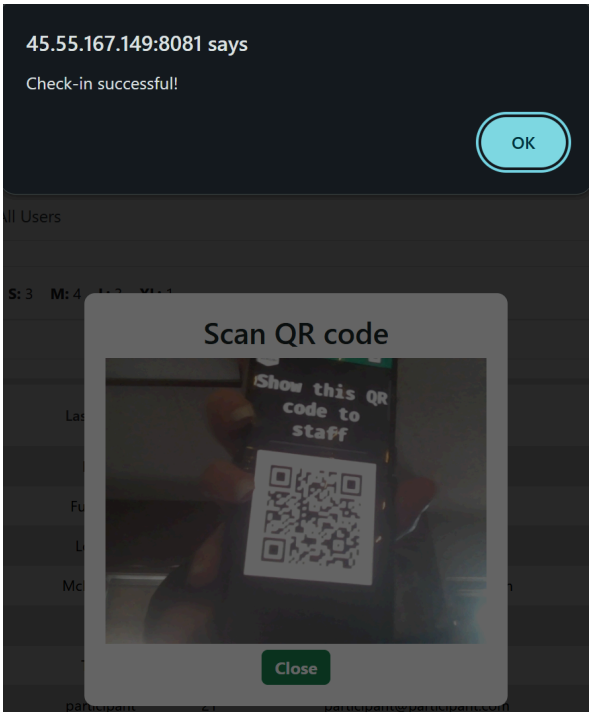


Figure 19. A participant was successfully checked in by scanning their QR code.

Event Management Page

Purpose: Enables staff to add, edit, and delete events (see Figure 14).

Design Components:

- Event list with filters
- Event creation/editing form
- Event overview panel

Supporting Data Models:

- Event

Inputs/Outputs:

- Input: Event name, dates, description
- Output: Updated event list and event details

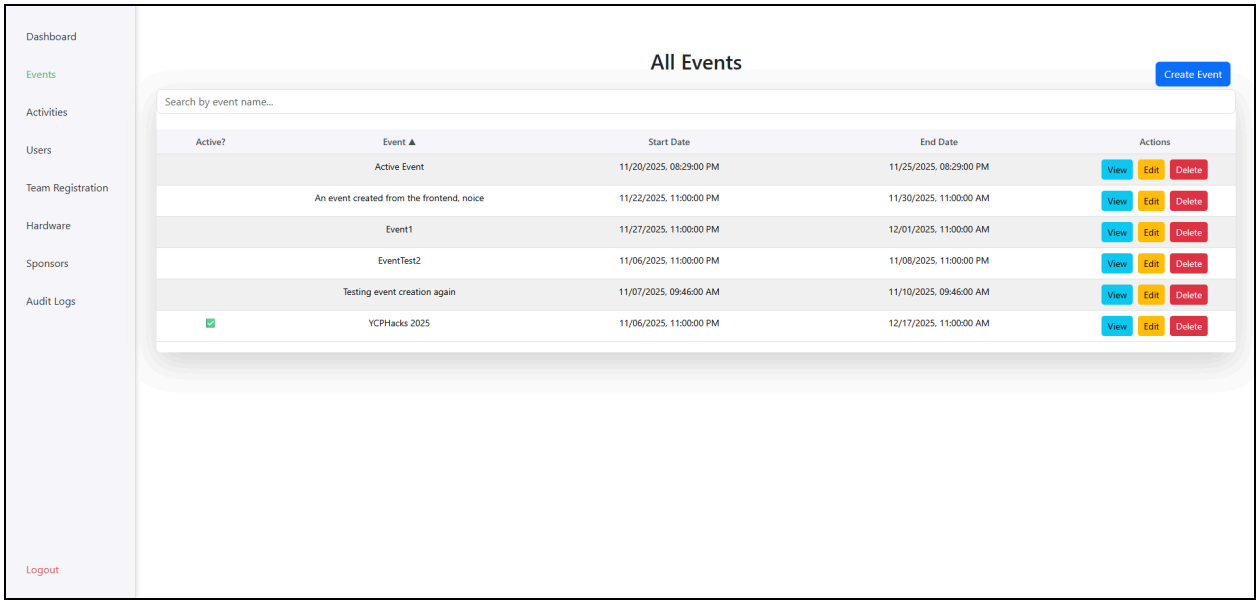


Figure 20: Shows the User Management Page which lists all users with various options to create, update, filter, export, or check in users.

Hardware Management Page (Admin)

Purpose: Provides full CRUD capabilities for hardware inventory, including the ability to upload images (see Figure 21).

Design Components:

- Hardware table
- Create/edit hardware form
- Image upload functionality
- Hardware detail modal

Supporting Data Models:

- Hardware
- HardwareImage

Inputs/Outputs:

- Input: Hardware details, image uploads

- Output: Updated inventory and associated images

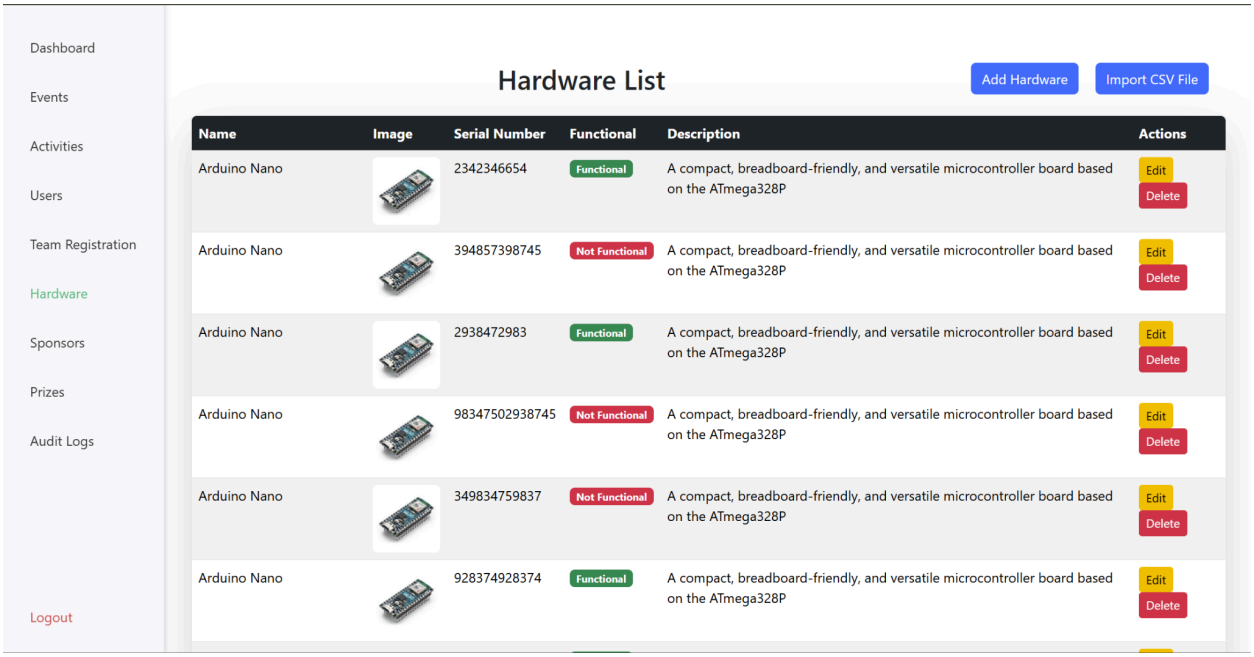


Figure 20: Shows the Hardware Page which displays and gives options to create, edit, or delete hardware.

Hack Category and Prize Management Page (Admin)

Purpose: Provides full CRUD capabilities for hack categories and the associated prizes..

Design Components:

- Hack Category table
- Prize Table
- Create/edit/remove hack category form
- Create/edit/remove prize form

Supporting Data Models:

- Hack Category
- Prize

Inputs/Outputs:

- Input: Hack Category and Prize details

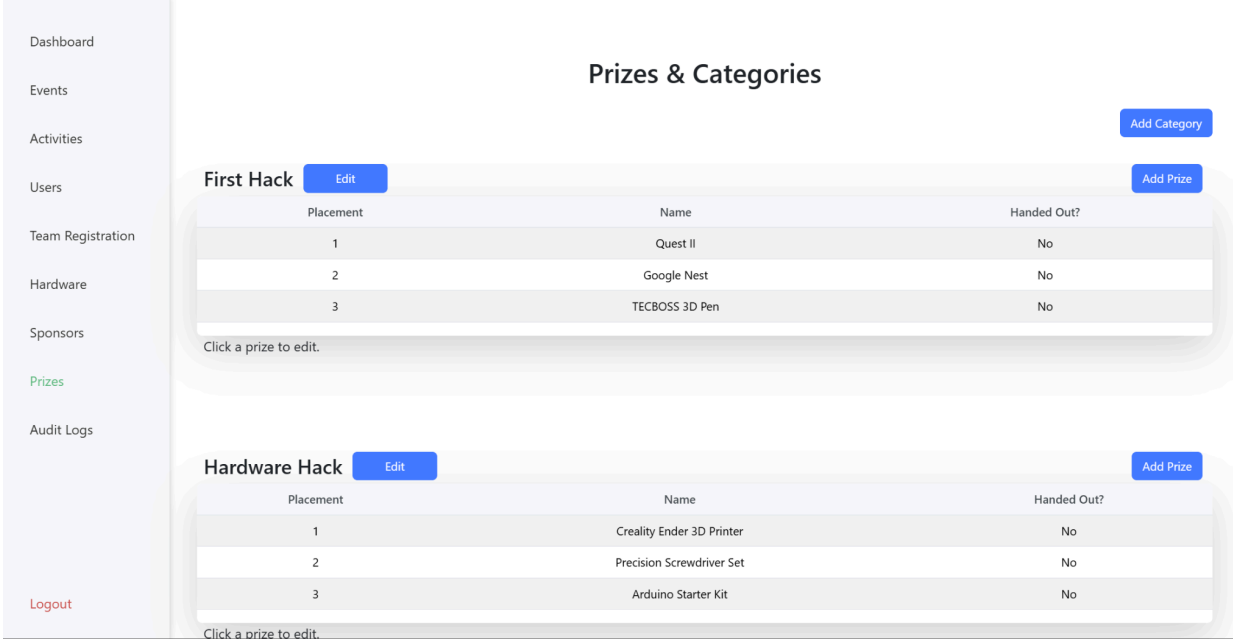


Figure 22: Shows the Prizes Page which lists all hack categories and the associated prize for the current active event, giving options to create, update, and delete them.

Activities Management (Admin)

Purpose: Allows administrators to schedule and manage hackathon activities (see Figure 16).

Design Components:

- Activity creation/editing form
- List of scheduled activities
- Search tool

Supporting Data Models:

- Activity
- EventActivity

Inputs/Outputs:

- Input: Activity details
- Output: Updated event schedule

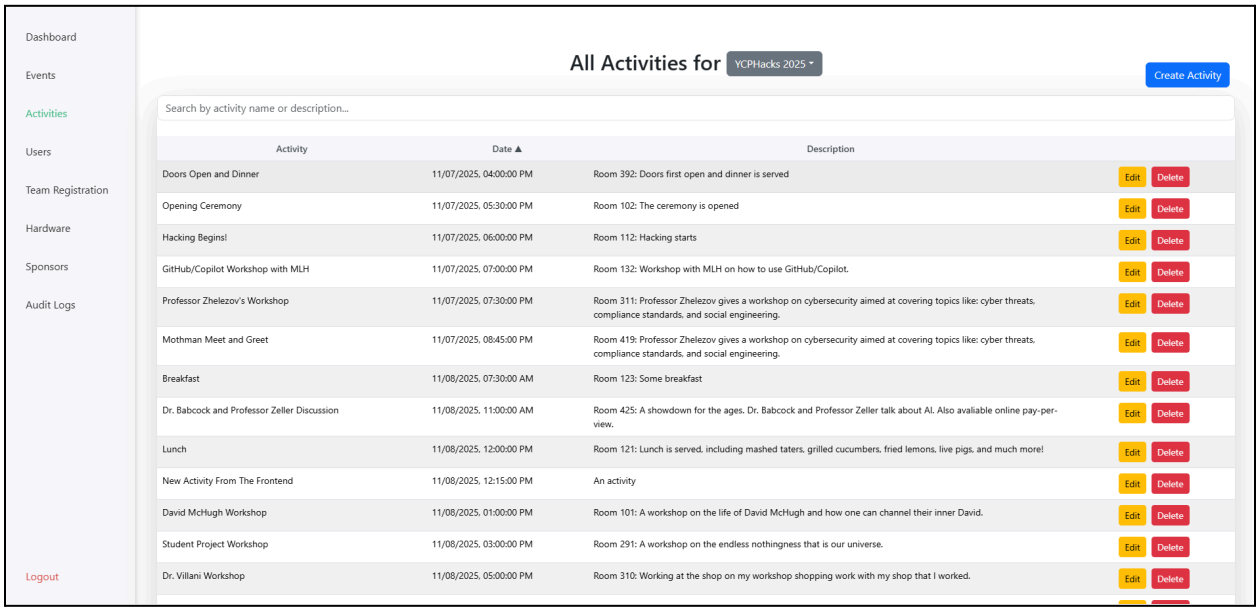


Figure 23: Shows the Activities Page which lists all activities, giving options to create, update, and delete them.

Teams Management (Admin)

Purpose: Enables staff to view and oversee participant teams for the event (see Figure 17).

Design Components:

- Team table with roster details
- Team detail modal
- Tools for updating team membership

Supporting Data Models:

- Team
- EventParticipant

Inputs/Outputs:

- Input: Updates to team information
- Output: Updated team lists and rosters

Figure 24: Shows the Team Registration Page which lists all teams and gives options for creating, updating, and deleting teams.

Audit Logs Page

Purpose: Provides a record of administrative actions for oversight, debugging, and security auditing (see Figure 25).

Design Components:

- Table of log events (timestamp, user, action)
- Filtering and search tools
- Pagination for large data sets

Supporting Data Models:

- AuditLog

Inputs/Outputs:

- Input: Search/filter parameters

- Output: Log entries

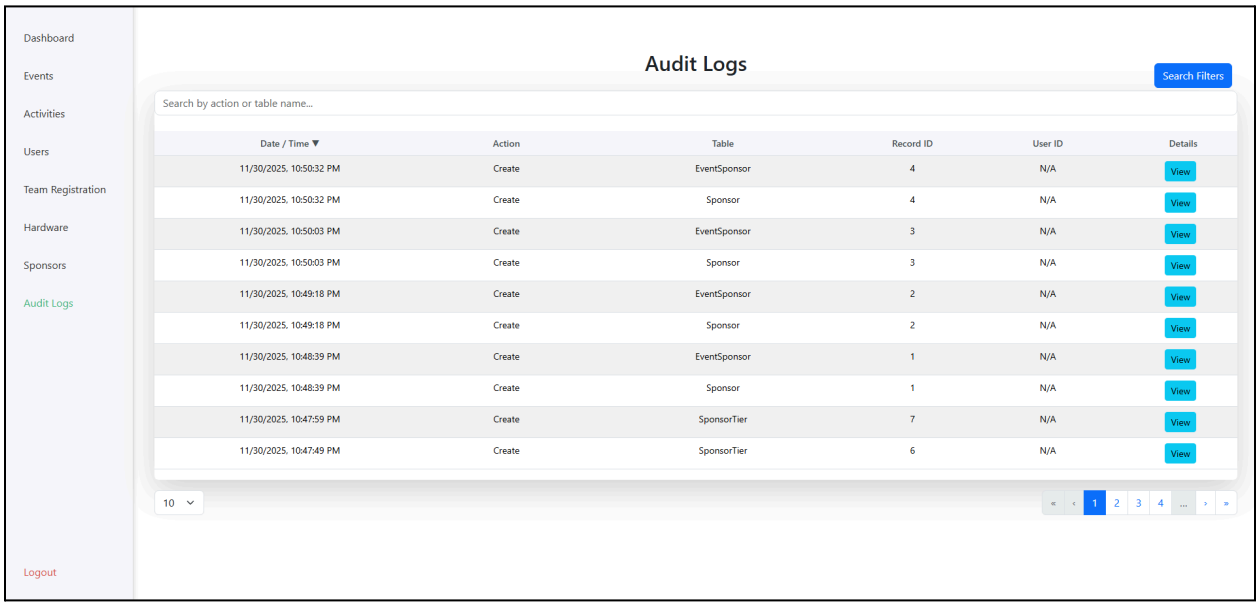


Figure 25: Shows the Audit Log Page which displays all audit logs and allows for pagination and filtering.

Sponsors Management Page (Admin)

Purpose: Enables staff to add, edit, and categorize sponsors for the event (see Figure 26).

Design Components:

- Sponsor table
- CRUD forms for sponsors
- Sponsor-tier relationships

Supporting Data Models:

- Sponsor
- SponsorTier
- EventSponsor

Inputs/Outputs:

- Input: Sponsor details, tier selection
- Output: Sponsor listing updates

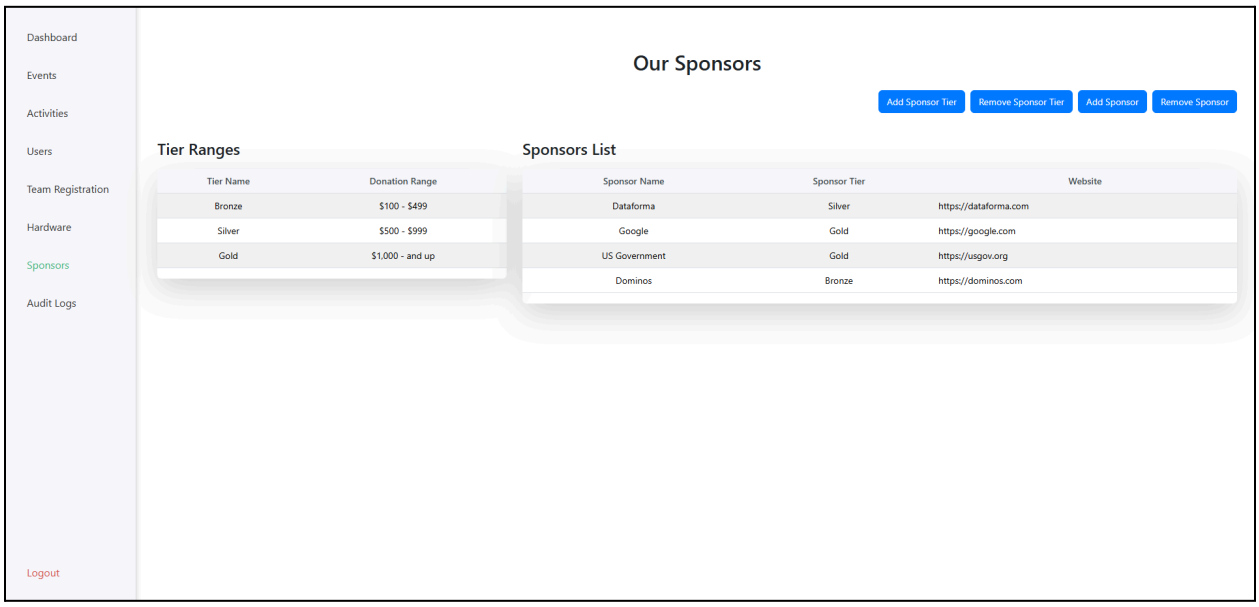


Figure 26: Shows the Sponsors page, listing both the tiers and sponsors, giving options to create, update, and delete tiers and sponsors.

Back End

The back end of the YCP Hacks web application is responsible for handling business logic, validating user input, and interacting with the MySQL database. It exposes a series of RESTful API endpoints that allow the frontend Vue.js application to retrieve, modify, and persist data across all participant and administrative workflows. All backend logic is implemented in Express.js, and the system uses Sequelize as the ORM tool to define models, manage migrations, and execute database queries without raw SQL.

Using Sequelize offers multiple advantages, including protection against SQL injection, automatic handling of associations, object-level validation, and a clear, maintainable structure for future development. Each database table is represented by its own Sequelize model, defined in a dedicated file to support modularity and ease of maintenance.

The database schema contains two primary categories of tables:

1. **Global tables**, which store data that can apply across multiple hackathon events (e.g., sponsors, hardware, users).

2. **Event-specific tables**, which store data tied to individual hackathon years (e.g., event participants, event sponsors, activities).

This separation ensures that the system can support multiple hackathons over multiple years without overwriting or altering historical data.

To maintain clean separation of concerns, all API routes, controller logic, and helper functions are organized into separate files. API endpoints never interact with the database directly; instead, they call service-layer functions that perform data validation, enforce permission checks, and communicate with Sequelize models. This layered approach enhances maintainability, readability, and long-term scalability.

The database schema follows a consistent naming convention in which table names are singular when each row in the table represents a single, distinct entity. For example, the User table stores individual user records, the Event table stores individual event records, and the Hardware table stores individual hardware items. This naming scheme improves clarity because the table name directly reflects the conceptual model of the entity being stored. This convention ensures that model definitions, SQL queries, and Sequelize associations remain intuitive and easy to understand for future developers and capstone teams.

Below is a detailed description of each table in the database schema, along with the rationale for its structure and how it interacts with other tables, as visually represented in the detailed schema diagram (Figure 27).

Figure 27. Database Schema

User

The User table stores all core information associated with individuals who interact with the system. Each user is uniquely identified by their email address and their system-wide user ID.

Users may act as participants, administrators, or staff members depending on their assigned role. A user's ID is referenced throughout the schema for team membership, event registration, hardware checkout, and administrative actions.

Hardware

The Hardware table stores all hardware assets available for checkout during the hackathon. It includes attributes such as name, description, quantity, and which participant currently has the item checked out. A reference to the staff member who processed the checkout is recorded for auditing purposes.

HardwareImage

Each hardware item can have multiple associated images. The HardwareImage table supports this by linking hardware items to image URLs using a one-to-many relationship. This allows the frontend to display photo galleries for individual hardware items.

Event

The Event table defines each hackathon event. Each record includes the event name, start date, end date, and whether editing is currently permitted. Most event-specific data (sponsors, participants, activities, teams, and categories) is stored in related tables.

EventParticipant

The EventParticipant table manages the many-to-many relationship between Users and Events. This table records which users are registered for which event and stores additional information that varies per event, such as participant status and associated team ID.

Activity

The Activity table represents possible activities (e.g., workshops, judging sessions, opening ceremonies). Activities may occur across multiple events. Each activity includes a title, description, and scheduled time.

EventSponsor

The EventSponsor table manages the relationship between sponsors and events. This design allows sponsors to change tiers, contribution amounts, or participation frequency across different hackathon years while maintaining a history of past sponsorships.

Prize

The Prize table defines individual prizes associated with a hackathon's category. Each prize includes a description, placement level (1st, 2nd, 3rd, etc.), and event linkage.

HackCategory

Hack categories represent competition tracks such as "Best Beginner Hack," "Gaming Hack," or "Best of Show." Each category may contain multiple associated prizes. Storing categories separately allows organizers to modify track offerings without altering historical data.

Team

The Team table stores information about participant teams for a specific event. This includes fields such as team name, GitHub project link, and presentation slide URL. Team membership is determined using the team ID stored in EventParticipants.

Sponsor

The Sponsor table stores globally reusable sponsor information, including name, logo image ID, and website URL. Sponsor details for individual events are connected through the EventSponsor table.

SponsorTier

Sponsor tiers (e.g., Gold, Silver, Bronze) are stored in the SponsorTier table. Each sponsor references a tier ID via the event-sponsor relationship. This makes it easy to introduce new sponsorship levels without changing the schema.

Image

The Image table stores image metadata and URLs used across the system. This table centralizes image management, enabling hardware, sponsors, events, or other components to reference image IDs rather than storing duplicate URLs. It also supports reuse of images across multiple events or years.

Analytics

The Analytics table stores discrete analytic events recorded throughout the system. This data supports future enhancements such as usage reports, trend analysis, and performance monitoring.

AuditLog

The AuditLog table captures administrative actions performed throughout the system. Each row logs a single action, including the staff member responsible, the resource affected, the type of action (create, update, delete), and the timestamp. Audit logging provides traceability, supports debugging, and strengthens administrative oversight by preserving a historical record of all system modifications.

Implementation

Front End

The front-end implementation forms the user-facing layer of the YCP Hacks platform and is responsible for rendering application data, handling user interactions, and communicating with the backend API in real time. The web application operates as a single-page application (SPA) built using Vue.js, allowing for fast navigation, modular component design, and dynamic updates without full page reloads.

Framework and Tools Selection

Vue.js was selected as the core front-end framework due to its strong component-based architecture, reactive data system, and seamless integration with modern build tooling. Vue's reactivity allows the interface to update immediately when backend data changes, enabling a responsive experience for both participants and administrators. The project also incorporates Vue Router for page-to-page navigation and Vuex for global state management, allowing authenticated user data, event information, and shared state to persist across components.

Bootstrap 5 was used selectively to provide responsive layout utilities and standard UI elements. While much of the styling was customized, Bootstrap's grid system and basic components accelerated layout development and ensured consistent behavior across device sizes.

Hot Module Replacement

Vite was selected for hot module replacement to keep the browser in sync with the code without needing a full page refresh. This allows for a faster development cycle by instantly showing changes made on the front end of the site to developers.

Component-Based Architecture

The front-end codebase is organized using a strongly component-driven architecture. Each page or functional unit (e.g., registration form, participant landing page, hardware listings, or admin pages) is encapsulated in its own Vue component. This approach improves maintainability, reduces duplication, and allows individual components to be tested and extended independently. These components standardized interactions across the application and made it significantly easier to implement new administrative tools and participant workflows.

Routing and Navigation

Vue Router was used to define and manage navigation between all pages of the application. This includes participant pages (e.g., registration, login, team management) as well as administrator pages (e.g., events, hardware, activities, sponsors). Routes were configured to enforce RBAC, ensuring that participants cannot access administrative tools.

Routes include:

- Public routes (landing, login, registration)
- Participant routes (teams, hardware listings)
- Admin routes (dashboard, events, hardware, users, audit logs, sponsors)

Route guards enforce authentication and permission checks before accessing protected pages.

API Integration

All front-end communication with the backend API was handled through the Vuex store.js files. When users perform operations (e.g., submitting a registration, editing hardware, or loading events) the front end issues an HTTP request to the Express backend, which responds with JSON data. The front end then updates the SPA based on the received data.

API integration improvements completed this year include:

- Connecting all admin pages to real backend data
- Replacing nearly all hard-coded values from last year
- Handling authentication tokens via Vuex
- Adding error handling and success notifications
- Ensuring consistent request/response structures
- Sending emails for email verification and password reset
- PDF generation of all the teams
- CSV parsing for data import

Styling and User Interface Design

While Bootstrap provided structural components, custom CSS and Vue styling were heavily used to match the application's design direction. Specifically for the participants page, styling is split into two separate sections. Coloring of webpages is handled through a global styling sheet which its use is meant only for coloring themes. The structure of the page is then defined within the local Vue components css. The team refined typography, spacing, interactive states, coloring, and mobile responsiveness throughout the application. Many administrative pages include tables, forms, and modals for CRUD operations, all of which were styled to remain consistent with last year's design but improved for clarity and usability.

Form Validation and Input Handling

Vue's reactivity and computed properties were leveraged to validate form inputs in real time. Registration fields, login credentials, hardware forms, event details, activity creation forms, and sponsor details all include front-end validation to prevent as many invalid submissions as possible. Any errors retrieved from the backend are displayed inline to guide users on why the submission failed.

Back End

The backend implementation provides all system logic, data validation, permissions enforcement, and persistent data storage for the YCP Hacks platform. It was developed using Express.js and Sequelize, communicating with a MySQL database running in Docker.

Throughout this semester, the backend underwent a substantial overhaul to support complete integration with the front end.

Model Development

Backend implementation begins by defining Sequelize models for each table in the schema. Each model specifies its fields, data types, and constraints. This includes definitions for entities such as User, Event, Team, Hardware, Sponsor, Activity, Analytics, and AuditLog.

Models were organized in individual files to improve maintainability. When a model represented a resource that might require separate logic for creation vs. retrieval, the team introduced secondary “DTO-model-like” structures to simplify validation and input handling.

Process Restarter

Nodemon was selected as a process restarter, when changes are detected in the back end the application is restarted automatically. This allows for a faster development cycle by instantly restarting the application to show changes made on the back end of the site to developers.

Associations and Relationships

After defining the models, associations were declared in the Sequelize index configuration file. These associations are critical for establishing the relationships between entities, enforcing referential integrity, and enabling complex data retrieval across the application.

The model structure enforces a powerful centralization around the Event entity, which serves as the primary hub for hackathon-specific data, ensuring data is properly scoped to its respective hackathon instance. These relationships, where the foreign key is eventId and the association uses onDelete: ‘CASCADE’, include:

- An Event can have many Event Participants
- An Event can have many specific Sponsors
- An Event is composed of many Activities
- An Event awards many Prizes

- An Event hosts many Teams
- An Event is judged against many Hack Categories

The diagram also specifies the crucial relationships for managing user participation and teams:

- A Team can have many Event Participants
- A User can have many roles as an Event Participant (allowing a single user to participate in multiple events)

These associations link media and other related entities:

- A Hardware item can have multiple images associated with it, using the alias ‘images’
- A Sponsor entity belongs to a specific Image (for their logo), referencing the foreign key sponsorImageId
- Sponsors belong to SponsorTiers

The diagram describes the relationship between hack categories and prizes:

- A Hack Category has many prizes

Sequelize automatically generates foreign keys based on the association definitions unless overridden by custom field names. This layer abstracts away complex SQL and ensures referential integrity through configurations like onDelete: ‘CASCADE’, which guarantees that when an Event is deleted, all associated records (Teams, Prizes, Activities, etc.) are automatically removed.

API Routes

Routes serve as the entry points for frontend communication. Each route file (e.g., UserRoutes.js, EventRoutes.js, HardwareRoutes.js) defines a set of endpoints that map to the service-layer functions (which are in the corresponding controller files). Routes enforce authentication and authorization through middleware to ensure only valid users can perform certain actions.

Examples include:

USER

- POST /users/register
    - Create a new user
- POST /users/login
    - Login for user
- POST/users/admin-login
    - Login for staff user
- POST/users/auth
    - Auth token
- GET /users/all
    - Gets all users
- GET/users/event/:eventId/staff
    - Gets all the staff for a specific event
- PUT/users/:id/checkin
    - Updated the checked in users
- PUT/users/:id
    - Updates a specific user

PRIZE

- POST/prize/create
    - Creates a new category
- GET/prize/by-event/:eventId
    - Gets all categories for a given event
- GET/prize/:id
    - Gets a prize by its unique id
- PUT/prize/update
    - Updates a specific prize

HACK CATEGORY

- POST/category/create

    - Creates a new hack category
- GET/category/by-event/:eventId
    - Gets all categories for a given event
- GET/category/:id
    - Gets a hack category by its unique id
- PUT/category/update
    - Updates a specific hack category
- DELETE/category/delete/:id
    - Deletes a given hack category by its unique id

EVENT

- POST /events/create
    - Create a new event
- POST/events/activity
    - Create a new activity
- POST/events/category
    - Create a new category
- GET/events/activity/:id
    - Gets activities from a specific event
- GET/events/all
    - Gets all the events
- GET/events/active
    - Gets all the events listed as active
- GET/events/:id
    - Gets a specific event
- GET/category/:id
    - Gets categories for a specific event
- PUT/events/update
    - Edit an existing event
- PUT/events/category
    - Edit an existing category

- PUT/events/activity
    - Edit and existing activity
- PUT/events/update
    - Edit the event detail of an existing event
- DELETE/events/:id
    - Delete an event
- DELETE/events/activity/:id
    - Delete activities for a specific event

UPLOAD

- POST/upload
    - Add an image
    - Mainly for digital ocean hookup

TEAM

- POST/teams/create
    - Creates a new team
- GET /teams/all
    - Gets all the teams by the event
- GET/teams/unassignedParticipants
    - Gets any participants not on a team
- GET /teams/:userId/team
    - Gets the users team status
- GET /teams/:teamId/project-details
    - Gets the teams project details
- PUT/teams/unassign
    - Updates the assigned and unassigned flag for a participant
- PUT/teams/:id
    - Update the team
- PUT /teams/:teamId/project-details
    - Updated the project details

- DELETE /teams/:id
    - Deletes a team

SPONSOR

- GET /sponsors/
    - Gets the list of sponsors
- GET /sponsors/:sponsorId/images
    - Get a specific sponsor image by the sponsor id
- GET/sponsors/by-event/:eventId
    - Get a sponsor by the event its associated with
- GET /sponsors/tiers
    - Get the list of sponsor tiers
- PUT/sponsors/:id
    - Update an existing event sponsor
- PUT /sponsors/tiers/:id
    - Update an existing sponsor tier
- POST /sponsors/
    - add the sponsor to an event
- POST /sponsors/tiers
    - Add a sponsor tier
- POST/sponsors/images/:id
    - Add an image to a sponsor
- DELETE/sponsors/:id
    - Delete a specific sponsor
- DELETE /sponsors/tiers/:id
    - Delete a specific sponsor tier
- DELETE /sponsors/images/:id
    - Delete a specific sponsor image

HARDWARE

- GET /hardware/

    - Gets all the hardware
- GET /hardware/admin
    - Gets all the hardware for the admin side
- GET /hardware/availability
    - Shows what is available in the list of hardware for participants
- GET /hardware/:id
    - Get hardware by id
- POST /hardware/add
    - Create a piece of hardware
- POST /hardware/image/add
    - Add the image for the new hardware
- PUT /hardware/update/:id
    - Update an existing piece of hardware
- DELETE /hardware/delete/:id
    - Delete any piece of hardware

AUDITLOG

- POST /auditlogs/search
    - Gets all logs

EMAIL

- GET / verify/verify-email
    - Updates the email verification status in the user database
- POST /verify/forgot-password
    - Sends an email with a reset password link
- POST /verify/reset-password
    - Updates the password of the user to the database

Route definitions were cleaned, refactored, and expanded significantly this year as the frontend grew.

Backend Validation and Security

The backend includes multiple layers of safety:

- Sequelize validation (length, required fields, format checks)
- Protected routes using middleware such as:
    - Token Authorization - The authenticity of JWT tokens are verified by checking if the signature is valid and if it's passed the time of expiration
    - Role Authorization - Checks the users role stored in the JWT tokens to prevent unwanted access to specific routes if the required role is not present in the token payload
    - Ownership of Requested User ID Authorization - Users can only request their own user ID (This is primarily used for only allowing users to reset their own password)
    - Sanitization of inputs to prevent improper types or invalid state changes using express-validator
- Email verification requires all users to check their email and click a link to confirm that their email is valid shown in figure 28
- Secure password handling is implemented using hashing and salted storage.

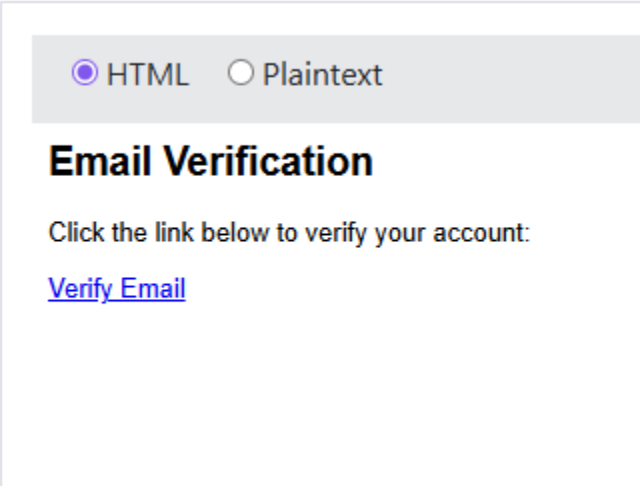


Figure 28. Email for account verification

Testing and Debugging

The CMS back end relies on Jest as the primary framework to ensure the platform's reliability, schema integrity, and protection against regressions when new features are added. The testing strategy is built around robust unit tests, with a total of 103 tests across 8 test suites, as shown in Figure 29, demonstrating comprehensive coverages of critical backend logic. Both the participant and administration front ends utilize Vitest to test and verify components are being rendered and that interactions with UI elements perform the correct actions. The participant page contains a total of 11 tests across 3 test suites in Figure 30 and the administrative page contains a total of 3 tests in a single test suite in Figure 31. These testing suites ensure reliability across both front ends of the application.

Framework and Coverage

Jest was chosen for its zero-configuration capabilities and parallel test execution, facilitating rapid and reliable testing of the Node.js back end. The tests specifically target:

- Service-Layer Functions to verify the core business logic returns the correct output
- Association Correctness to validate the integrity and relationships within the database schema
- Utility Functions to ensure general helper functions operate reliably

Vitest was also chosen for its Vite-native integration and optimized build processes for testing of the Vue.js front ends. These test target:

- Component Rendering to ensure components such as the nav bar are showing up on the site
- Component Interactions to test that interactable buttons when clicked perform the expected action

The successful execution of these tests, as shown in Figure 29, 8 suites and 103 tests passed, Figure 30, 3 test files and 11 tests passed, and Figure 31 confirms the system's current stability and functionality.

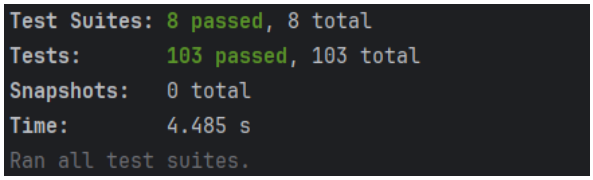


Figure 29. Results of back end tests running

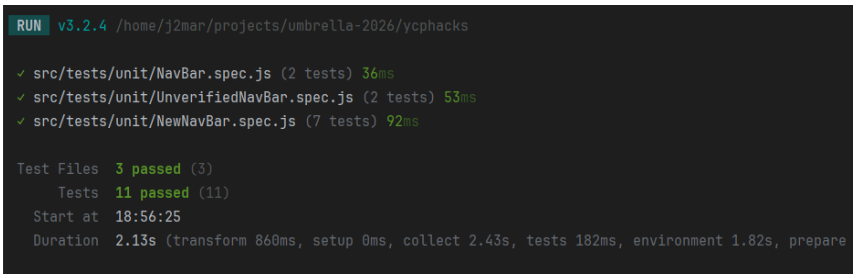


Figure 30. Result of front end tests running for participant site

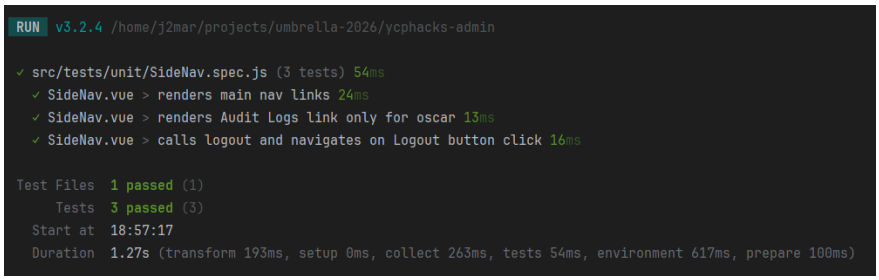


Figure 31. Result of front end tests running for administration site

Methodology: Detailed Unit Testing

To ensure the quality of the entire application, the team adhered to strict unit testing best practices. Using the example of the EventParticipantController tests, which manages the user-to-team assignments, we can illustrate the methodology applied across the system:

1) A core principle was to isolate the controller logic from external dependencies. The tests use jest.mock() to replace the Repository Layer (e.g., EventParticipantsRepo, UserRepo) with mocks. This guarantees that tests only verify the controller's logic (input validation, data formatting, and business decisions) without relying on live database operations.

2) Testing extended beyond successful operations to include detailed error and edge-case validation across all major API routes:
    a) **Input Validation:** Tests verify that the controller handles scenarios where required parameters are missing (e.g., returning a 400 status).
    b) **Data Integrity Checks:** The logic for retrieving participants was specifically tested to ensure that users marked as banned are correctly filtered out before being displayed to staff.
    c) **Robust Error Handling:** Every critical route was tested to ensure that repository failures (database errors) are caught and handled gracefully, typically by returning a 500 status code with an appropriate error message and suppressing non-critical console output during the failure.
3) Tests rigorously verify the outcome, including confirmation that the correct HTTP status code (e.g. 200, 400, 404, 500), and using toHaveBeenCalledWith() to verify the controller made the correct call with the expected parameters to the mocked methods.

This structured and detailed approach ensures that all critical paths in the backend are validated, providing a high degree of confidence in the platform's overall reliability.

Docker Integration

Each aspect of the project is separately containerized using docker as shown in Figure 32. The ycphacks-admin container stores the front end code for the administrative site, ycphacks contains the front end code for the participants site, mysql-cron-backup has the backup service used for the database, api stores the api and backend code, and db is the container for the database.

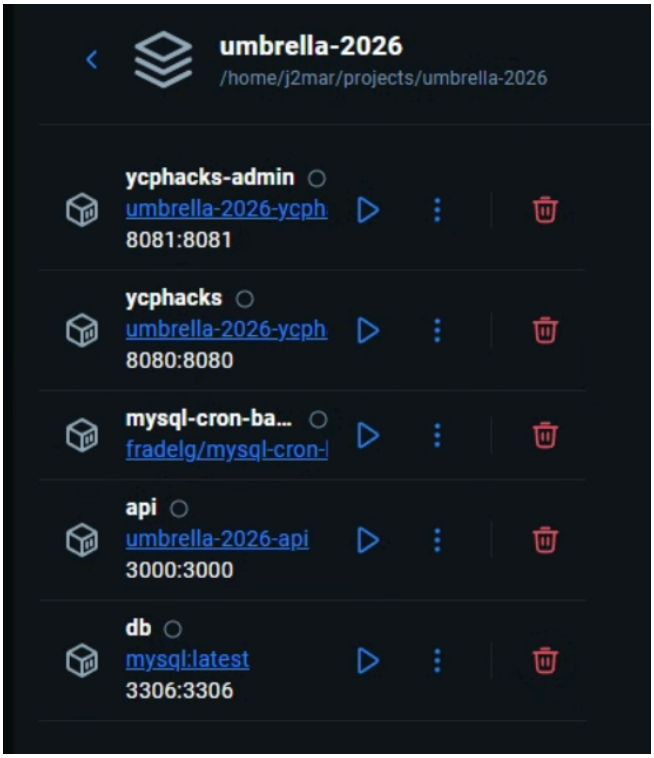


Figure 32. Containers created for each aspect of the project

Running the entire application with docker containers ensures that:

- Development environments are consistent and identical for all team members
- Predictable runtime environments
- Easy rebuilding of the entire application

There are .env variables set up as well as .dockerignore files, docker-compose.yml, and Dockerfile files for every submodule in the project. When the database container is shut down, the mysql-cron-backup container will detect this and create a local backup of the data inside of the database.

Deployment

The application has now been configured to work in a deployment environment. A DigitalOcean droplet is being used to host the container images of the project. This allows for the application to now be publicly accessible to anyone. A separate droplet called ycphacks-staging

was created so that this application can be tested alongside the current site that is being used for the hackathon event.

Deployment is done by using a CI/CD pipeline that is handled by GitHub Actions. Once a push is made to the main branch of the Umbrella repository, GitHub Actions will perform the following actions in the droplet-deployment.yaml and can be viewed in figure 33:

- Build and push the docker image for the ycphacks, ycphacks-admin, and ycphacks-api containers
- Store the latest images of these containers inside of a GHCR shown in figure 34
- Login to DigitalOcean and navigate to the project folder
- Login to GHCR and pull the latest images for the containers
- Look at the docker-compose.yml to pull the latest version of the remaining services ex. my-sql-cron-backup
- Start up all of the docker images in the droplet

Figure 35 shows that the web application is now accessible publicly by using the public IP of the Digital Ocean Droplet.

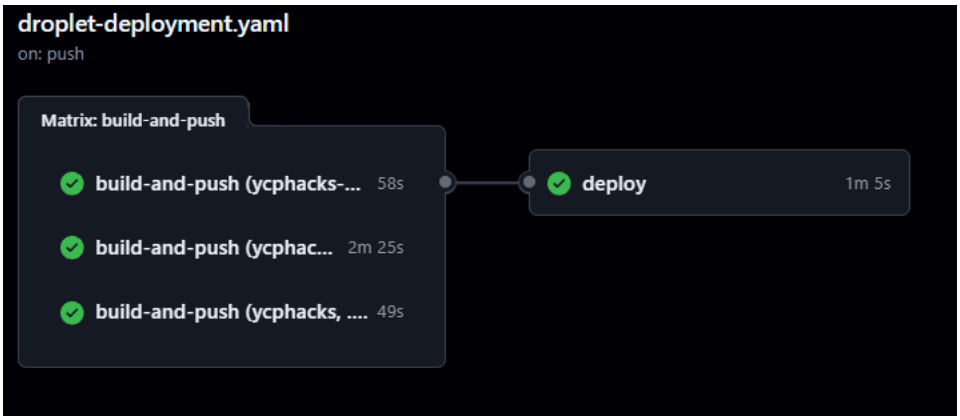


Figure 33. Diagram showing the ycphacks, ycphacks-admin, and ycphacks-api containers being built and pushed before deployed to the DigitalOcean Droplet

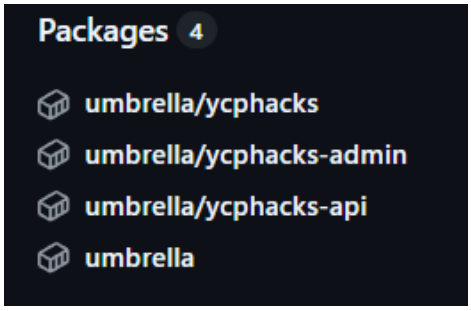


Figure 34. Shows the container images stored in the GHCR for the ycphacks, ycphacks-admin, and ycphacks-api containers

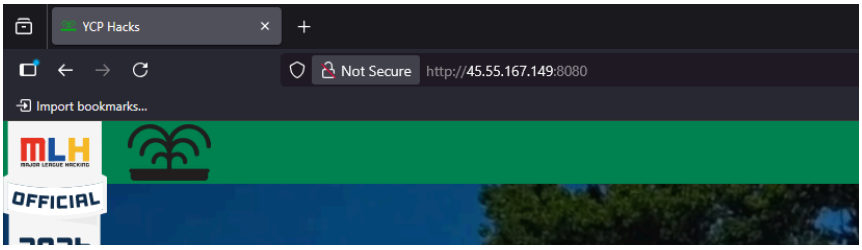


Figure 35. Shows the participants landing page being accessed using the public IP of the Digital Ocean Droplet

Future Work

As with all multi-year software engineering projects, the YCP Hacks platform remains a work in progress. While this year's team delivered major improvements–including full integration with the backend, expanded administrative tools, team creation, activities management, hardware improvements, audit logging, and a significantly more maintainable database schema–there are several key opportunities for enhancement and expansion. The following outlines both long-term strategic improvements and short-term onboarding tasks for future development teams.

Front End

Several areas of the front-end can be enhanced to improve usability, maintainability, and overall user experience.

UI/UX Improvements

While the current interface is functional, future teams should focus on refining the visual consistency and polish of the platform. Important improvements include:

- Improved responsiveness, ensuring all pages behave correctly on tablets and mobile devices especially the admin
- Fix participant-page-sponsor-image bug, problems with sponsors not always showing up when connecting to the hosted version
- Team creation through individual users, decentralize team creation from staff to users. However, still requiring staff to accept changes to team and project names and links. As well as adding functionality for teams to upload a pdf or pptx for their presentations as an alternative to linking a shared online presentation editor.
- Add visual navbar indication that a user on a particular page

Advanced Features to Add

Future teams may consider implementing interactive and workflow-oriented front-end features, including:

- Real-time updates via WebSockets (e.g., hardware status, checked in users, admin dashboard)
- Notification system for announcements, reminders, admin alerts, and participant team requests
- Improved image handling UI for sponsors and hardware
- Data Filtering to sort through data based on user requests

Accessibility Enhancements

To ensure the platform is inclusive and usable by all:

- Add ARIA roles, proper semantic tags, and keyboard navigation support

- Ensure color contrast meets WCAG standards
- Improve focus states and navigation for form-heavy pages

Performance Optimization

Future improvements may include:

- Implementing lazy loading for images and large components
- Code-splitting Vue components to reduce bundle size
- Caching common API responses using Vuex or browser storage

Refactoring Opportunities

To improve maintainability and system consistency:

- Introduce more unit tests for Vue components to increase reliability

Back End

While this semester's team made improvements to the backend, there are several areas where future work is continuously required.

Security Improvements

Future teams should prioritize strengthening platform security:

- Rework the email token validation system to avoid users accessing the API endpoint directly
- Implement a reverse-proxy to to manage incoming requests and provide rate-limiting

Database and Sequelize Enhancements

While the database has been heavily improved, future work may include:

- Writing Sequelize seeders for test/demo data
- Adding additional database constraints for improved integrity

- Improving query optimization for high-volume reporting or analytics
- Update the Hardware table to separate the consistent information between the same hardware. e.g. images and names

Administrative Functionality

Enhancements to administrative tools will improve staff efficiency:

- Implement multiple staff levels for specific event permissions and restrictions

Deployment and Infrastructure

As the system evolves, future teams may improve deployment processes:

- Finish the current working CI/CD pipeline to include automatic testing and rejection based on success of said tests
- Create integrated testing to test the interactions between grouped modules on a testing focused database
- Implement environment-based configuration for development, staging, and production

Recommended Startup Tasks for the Next Team

These are tasks that are small enough to start, but valuable for familiarizing new people with the system.

Front End Startup Tasks

1. Add background video to the Admin Login page

The registration page uses a video background; the login page does not. This small UI task will help the new team learn project structure and styling conventions.

2. Fix the YCP Hacks Logo offset bug

The YCP Hacks fountain logo in the left corner of the participant site incorrectly scales its offset within mobile devices. This is a task that is effective in learning the CSS conventions used within the frontend sites.

3. Finish the light and dark mode

The admin site currently does not support a dark mode, and the participant site dark mode should be redesigned for a better user experience as well as requiring consistency between pages. This is another useful way to begin work on the frontend portion of this project and informs on the conventions used with cross-page HTML and CSS.

Back End Startup Tasks

1. Install Postman (or Thunder Client) and manually test all API endpoints

This helps new team members understand how data flows through the system while retesting important parts of the system.

3. Improve validation on key models

A manageable task that teaches Sequelize schema rules and error handling.

4. Improve AuditLog

Track the id of the user who made the changes and implement search term functionality. This builds familiarity with service functions and backend logic.

References

[1] GitHub, “GitHub.com Help Documentation,” *docs.github.com*, 2024. https://docs.github.com/en

[2] Vue, “Introduction | Vue.js,” *vuejs.org*, 2022. https://vuejs.org/guide/introduction.html

[3] OpenJS Foundation, “Express - Node.js web application framework,” *Expressjs.com*, 2017. https://expressjs.com/

[4] Bootstrap, “Bootstrap,” *Getbootstrap.com*, 2022. https://getbootstrap.com/

[5] Axios, “Getting Started | Axios Docs,” *axios-http.com*, 2023. https://axios-http.com/docs/intro

[6] “Overview | Vue CLI,” *cli.vuejs.org*. https://cli.vuejs.org/guide/

[7] DigitalOcean, LLC, “DigitalOcean – The developer cloud,” *DigitalOcean*, 2022. https://www.digitalocean.com/

[8] Node.js, “Node.js,” *Node.js*, 2023. https://nodejs.org/en

[9] Vite, “Vite,” *vitejs*, 2024. https://vite.dev/

[10] “dotenv,” *npm*. https://www.npmjs.com/package/dotenv

[11] Docker, “Enterprise Application Container Platform | Docker,” *Docker*, 2024. https://www.docker.com/

[12] W3 Schools, “JavaScript Tutorial,” *W3schools.com*, 2019. https://www.w3schools.com/Js/

[13] “Overview | Vue CLI,” *cli.vuejs.org*. https://cli.vuejs.org/guide/

[14] “Wiki.js,” *js.wiki*. https://js.wiki/

[15] npm, “npm | build amazing things,” *Npmjs.com*, 2019. https://www.npmjs.com/

[16] What is RBAC? (2025). Snowflake.com. https://www.snowflake.com/en/fundamentals/rbac/

[17] OpenJS Foundation. (2017). *Express - Node.js web application framework*. Expressjs.com. https://expressjs.com/

[18] Vue Router. (n.d.). Router.vuejs.org. https://router.vuejs.org/

[19] Getting Started | *Vuex*. (n.d.). Vuex.vuejs.org. https://vuex.vuejs.org/guide/

[20] “Working with the Container registry,” *GitHub Docs*. https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry

[21] “A Beginner’s Guide to Unit Testing with Vitest | Better Stack Community,” *betterstackhq*, 2016. https://betterstack.com/community/guides/testing/vitest-explained/

[22] “fradelg/mysql-cron-backup - Docker Image,” *Docker.com*, 2026. https://hub.docker.com/r/fradelg/mysql-cron-backup (accessed Apr. 30, 2026).

[23] GeeksforGeeks, “Generate a QR code in Node.js,” *GeeksforGeeks*, Jan. 04, 2021. https://www.geeksforgeeks.org/node-js/generate-a-qr-code-in-node-js/ (accessed Apr. 30, 2026).

[24] “Getting Started · express-validator,” *Github.io*, 2019. https://express-validator.github.io/docs/

[25] “validator,” *npm*. https://www.npmjs.com/package/validator

[26] “Nodemailer. (n.d.).” *Nodemailer documentation*. https://nodemailer.com/

[27] “Node.js stream module Node.js. (n.d.).” *Stream API documentation*. https://nodejs.org/api/stream.html

[28] “csv-parser” npm, Inc. (n.d.). *csv-parser*. https://www.npmjs.com/package/csv-parser

[29] “Puppeteer” Google Chrome Developers. (n.d.). *Puppeteer documentation*. https://pptr.dev/

[30] J. Erickson, “What is MySQL?,” *Oracle*, Aug. 29, 2024. https://www.oracle.com/mysql/what-is-mysql/

[31] “GitBot,” *Discord Bots*, 2020. https://discord.bots.gg/bots/761269120691470357 (accessed May 01, 2026).

[32] “nodemon,” *npm*. https://www.npmjs.com/package/nodemon

[33] Sequelize ORM, “Sequelize,” Sequelize ORM, 2019. https://sequelize.org/

[34] GeeksforGeeks, “Testing with Jest,” *GeeksforGeeks*, Jan. 30, 2020. https://www.geeksforgeeks.org/javascript/testing-with-jest/